

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ



LUẬN VĂN TỐT NGHIỆP

THIẾT KẾ HỆ THỐNG GIÁM SÁT
NĂNG LƯỢNG ĐIỆN

GVHD: Th.S Bùi Quốc Bảo

SVTH: Nguyễn Xuân Phát

MSSV: 2212529

TP. HỒ CHÍ MINH, THÁNG 3 NĂM 2026

ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc.

Số: /BKDT
Khoa: Điện – Điện tử
Bộ Môn: Điện tử

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

- HỌ VÀ TÊN:** Nguyễn Xuân Phát **MSSV:** 2212529
- NGÀNH:** ĐIỆN TỬ - VIỄN THÔNG **LỚP:** DD22DVT3
- Đề tài:** Thiết kế hệ thống giám sát năng lượng điện.
- Nhiệm vụ (Yêu cầu về nội dung và số liệu ban đầu):**
 - Thiết kế và thi công phần cứng cho hệ thống.
 - Thiết kế và phát triển phần mềm nhúng cho hệ thống.
 - Thiết kế và phát triển trang Web theo dõi cho hệ thống.
- Ngày giao nhiệm vụ đồ án:** 05/01/2026
- Ngày hoàn thành nhiệm vụ:**
- Họ và tên người hướng dẫn:** Th.S Bùi Quốc Bảo
Phần hướng dẫn
Thiết kế phần cứng.
Phát triển phần mềm nhúng.
Phát triển trang Web.

Nội dung và yêu cầu ĐATN đã được thông qua Bộ Môn.

Tp.HCM, ngày tháng năm 2026

CHỦ NHIỆM BỘ MÔN

NGƯỜI HƯỚNG DẪN CHÍNH

PHẦN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ luận văn:

Lời cảm ơn

Để hoàn thành báo cáo này, trước hết em xin gửi lời cảm ơn sâu sắc đến tập thể phòng Lab 209B3 và thầy Bùi Quốc Bảo, người đã trực tiếp hướng dẫn em trong luận văn tốt nghiệp này, cũng như thầy, cô giảng viên trong Khoa Điện – Điện tử, Trường Đại học Bách Khoa – ĐH Quốc gia TP. HCM, đã truyền đạt kiến thức và kinh nghiệm quý báu cho em trong suốt quá trình học tập và rèn luyện của em tại trường. Trong quá trình làm luận văn, em đã gặp không ít khó khăn, còn thiếu sót nhiều về mặt kiến thức thực tế. Nhưng với sự giúp đỡ, hỗ trợ từ anh Cao Văn Hiếu, anh Huỳnh Thanh Sang trong việc thiết kế phần cứng và những người bạn tại phòng Lab 209B3 trong việc chia sẻ kiến thức với em, và đặc biệt là sự giúp đỡ, tư vấn nhiệt tình từ thầy Bùi Quốc Bảo chia sẻ những kiến thức và kinh nghiệm của thầy để em có thể hoàn thành tốt luận văn của mình và tích lũy thêm cho bản thân được nhiều những kinh nghiệm thực tế, kiến thức hữu ích cho bản thân, cải thiện kỹ năng tìm kiếm thông tin trong quá trình thực tập để hoàn thành nhiệm vụ được thầy giao. Dù bản thân đã rất cố gắng để hoàn thành tốt các nhiệm vụ được giao nhưng không thể tránh khỏi những sai sót nên em rất mong nhận được sự thông cảm, đóng góp ý kiến của quý thầy cô.

Cuối cùng, em xin kính chúc quý thầy cô sức khỏe, luôn thành công trong công việc và cuộc sống. Em xin chân thành cảm ơn!

TP.Hồ Chí Minh, ngày 25 tháng 3 năm 2026

Nguyễn Xuân Phát

Tóm tắt luận văn

Trong bối cảnh hiện nay, các doanh nghiệp đang có xu hướng công nghiệp hóa hiện đại hóa các quy trình đóng gói sản xuất, các thiết bị hiện đại có khả năng tự vận hành cũng vì thế mà phát triển ngày càng nhiều theo nhu cầu của thị trường. Các loại máy móc này hoạt động với tần suất liên tục suốt hàng giờ vì thế không tránh khỏi các tình trạng sai số do các linh kiện tạo ra, hỏng hóc trong quá trình vận hành do các yếu tố khách quan cũng như là chủ quan. Để có thể đảm bảo được quy trình sản xuất diễn ra đúng với kế hoạch của công ty thì công tác vận hành và bảo trì các loại máy móc cỡ lớn này cần được thực hiện thường xuyên để có thể kịp thời phát hiện ra lỗi và sửa chữa.

Từ đó, các thông số về điện áp hoặc dòng điện sai lệch như thế nào so với các thông số ban đầu mà nhà sản xuất máy móc đưa ra sẽ được thu thập và ghi chép lại sau đó được tính toán một cách cẩn thận và đưa ra hướng giải quyết phù hợp nhất. Thì hiện nay tại nhiều doanh nghiệp đã áp dụng công nghệ đo lường từ xa và gửi dữ liệu đo đạc về cho máy chủ bằng gateway giúp tiết kiệm được thời gian, công sức và tiền bạc. Tại các doanh nghiệp điện lực hiện nay đã áp dụng công nghệ này để ứng dụng vào đo lường giá trị điện áp tại các tủ điện và gửi về phòng điều khiển từ đó các kỹ sư sẽ theo dõi một cách trực quan về điện áp của từng tủ điện tại cùng một thời điểm, giảm rủi ro nguy hiểm do điện áp cao khi tiếp xúc tại nơi đặt tủ điện, từ đó có thể phát hiện sớm được các bất thường để đưa ra hướng giải quyết.

Mục lục

1	Giới thiệu	7
1.1	Tổng quan về đề tài	7
1.2	Tình hình nghiên cứu trong và ngoài nước	8
1.3	Nhiệm vụ đề tài	9
2	Cơ sở lý thuyết	10
2.1	Tìm hiểu về lý thuyết về giao thức Modbus	10
2.1.1	Khái quát về giao thức Modbus	10
2.1.2	Tìm hiểu lý thuyết về giao thức Modbus RTU	11
2.1.2.1	Tổng quan về giao thức Modbus RTU	11
2.1.2.2	Tìm hiểu chuẩn vật lý RS485	12
2.1.2.3	Tìm hiểu khung truyền dữ liệu Modbus RTU	13
2.1.3	Khái niệm về giao thức Modbus TCP/IP	15
2.1.3.1	Tổng quan về giao thức Modbus TCP/IP	15
2.1.3.2	Tìm hiểu về khung truyền dữ liệu Modbus TCP/IP	16
2.1.4	Tìm hiểu Function Codes trong giao thức Modbus	18
2.2	Tìm hiểu về giao thức Ethernet	19
2.2.1	Khái quát và các đặc của giao thức Ethernet	19
2.2.2	Tìm hiểu về cấu trúc gói tin Ethernet	20
2.2.3	Cách thức đóng gói gói tin Ethernet theo mô hình OSI	22
2.2.3.1	Tìm hiểu về mô hình OSI	22
2.2.3.2	Cách thức đóng gói tin Ethernet	23
2.3	Tìm hiểu về giao thức truyền thông MQTT	24
2.4	Tìm hiểu về các kiến thức liên quan tới trang web	25
2.5	Tìm hiểu về Database Postgres	26
2.6	Khái niệm về API RESTful và API Websocket	28
2.6.1	Khái niệm và đặc điểm về API RESTful	28
2.6.2	Khái niệm và đặc điểm về API Websocket	29
2.7	Tìm hiểu về phần mềm Postman	31
2.7.1	Khái niệm và Postman	31
2.7.2	Các đặc điểm và vai trò chính của Postman	31
3	Kết quả thực hiện	33
3.1	Thiết kế phần cứng	33
3.1.1	Tổng quan kiến trúc hệ thống	33
3.1.2	Các luồng tín hiệu có trong hệ thống	36
3.1.3	Chi tiết thiết kế phần cứng	37
3.1.3.1	Thiết kế khối vi điều khiển	37
3.1.3.2	Thiết kế khối nguồn	38

3.1.3.3	Thiết kế khối Ethernet	40
3.1.3.4	Thiết kế khối giao tiếp RS485	42
3.1.3.5	Thiết kế khối EEPROM và RTC	44
3.1.3.6	Thiết kế khối giao tiếp với người dùng	46
3.1.4	Kết quả thực hiện phần cứng thực tế	47
3.1.4.1	Kết quả thiết kế phần cứng trên phần mềm	47
3.2	Thiết kế phần mềm	53
3.3	Kết quả thu được	53
4	Kết luận và hướng phát triển	54
4.1	Kết luận	54
4.2	Hướng phát triển	54

Danh sách hình vẽ

2.1	Tổng quan về giao thức Modbus	10
2.2	Tổng quan về giao thức Modbus RTU	11
2.3	Tổng quan về chuẩn vật lý RS485	12
2.4	Sơ đồ kết nối RS485 với điện trở đầu cuối 120Ω	13
2.5	Khung truyền tổng quát của Modbus RTU	13
2.6	Khung truyền Master gửi Request cho Slave trong Modbus RTU . .	14
2.7	Khung truyền Slave gửi Response cho Master trong Modbus RTU .	14
2.8	Khung truyền Slave gửi báo lỗi cho Master trong Modbus RTU . . .	15
2.9	Tổng quan về giao thức Modbus TCP/IP	15
2.10	Khung truyền dữ liệu tổng quát của Modbus TCP/IP	16
2.11	Thành phần của một gói tin Modbus TCP/IP khi được gửi đi	17
2.12	Khung truyền yêu cầu Server gửi đến Client trong Modbus TCP/IP	17
2.13	Khung truyền phản hồi Client gửi đến Server trong Modbus TCP/IP	17
2.14	Khung truyền báo lỗi Client gửi đến Server trong Modbus TCP/IP .	18
2.15	Tổng quan về mạng Ethernet	19
2.16	Cấu trúc của gói tin Ethernet	20
2.17	Cấu trúc tổng quát của mô hình OSI	22
2.18	Quy trình đóng gói gói tin Ethernet	23
2.19	Mô hình giao tiếp MQTT	24
2.20	Tổng quan về các thành phần của một trang web	26
2.21	Hệ quản trị cơ sở dữ liệu Postgres	27
2.22	Tổng quan về RESTful API	28
2.23	Tổng quan về Websocket API	30
2.24	Logo của Postman	31
3.1	Sơ đồ khối tổng quát thực hiện phần cứng	33
3.2	Sơ đồ khối phân bố nguồn trong hệ thống	34
3.3	Sơ đồ khối luồng dữ liệu trong hệ thống	36
3.4	khối vi điều khiển	37
3.5	khối nạp code và Debug	37
3.6	khối kiểm tra nguồn cho vi điều khiển	38
3.7	khối lọc nguồn đầu vào cho vi điều khiển	38
3.8	khối nguồn đầu vào	38
3.9	Khối nguồn hạ áp 5V	38
3.10	Khối nguồn hạ áp 3.3V	39
3.11	khối nguồn cách ly 5V	39
3.12	khối nguồn Pin rời cho mạch RTC	39
3.13	khối vi điều khiển cho Ethernet	40
3.14	khối cổng kết nối cho Ethernet	40

3.15 khối lọc nguồn đầu vào cho Ethernet	41
3.16 khối thạch anh và DIP Switch cho Ethernet	41
3.17 khối xử lý tín hiệu RS485	42
3.18 khối cách ly RS485	43
3.19 khối cổng giao tiếp RS485	43
3.20 khối lọc nguồn đầu vào và LED báo nguồn	43
3.21 khối bộ nhớ EEPROM	44
3.22 khối RTC	44
3.23 khối Pull up cho đường truyền I2C	45
3.24 khối lọc nguồn đầu vào cho EEPROM và RTC	45
3.25 khối nút nhấn tương tác với người dùng	46
3.26 màn hình LCD hiển thị	46
3.27 phần đi dây ở Top Layer	47
3.28 phần đi dây ở Bottom Layer	48
3.29 phần đặt các symbol ở Top Layer	49
3.30 mô hình mạch 3D khi chưa đặt linh kiện ở Top Layer	50
3.31 mô hình mạch 3D khi chưa đặt linh kiện ở Bottom Layer	51
3.32 mô hình mạch 3D khi đặt linh kiện ở Top Layer	52
3.33 mô hình mạch 3D khi đặt linh kiện ở Bottom Layer	53

Danh sách bảng

2.1	Một số Function Codes phổ biến trong giao thức Modbus RTU . . .	18
2.2	Các phương thức HTTP và hành động tương ứng	29

CHƯƠNG 1

Giới thiệu

1.1 Tổng quan về đề tài

Trong kỷ nguyên Công nghiệp 4.0, việc giám sát và quản lý năng lượng điện hiệu quả đã trở thành yêu cầu cấp thiết đối với các doanh nghiệp nhằm tối ưu hóa chi phí vận hành và đảm bảo an ninh năng lượng. Lĩnh vực hệ thống nhúng và Internet of Things (IoT) đóng vai trò then chốt trong việc kết nối các thiết bị đo lường truyền thống với hạ tầng mạng hiện đại. Tuy nhiên, một thách thức lớn hiện nay là sự không đồng nhất về giao thức truyền thông giữa các thiết bị hiện trường (thường sử dụng thông qua chuẩn RS485) và các hệ thống quản lý trung tâm (thường sử dụng Ethernet/WiFi). Do đó, việc nghiên cứu và phát triển các bộ chuyển đổi giao thức (Gateway) có khả năng thu thập, xử lý và truyền tải dữ liệu lên các hệ thống quản lý từ xa theo thời gian thực và là một hướng nghiên cứu có tính ứng dụng cao và thực tiễn.

Mục tiêu chính của đề tài này là nghiên cứu giải pháp thiết kế một Gateway dựa trên vi điều khiển ESP32 để giám sát các thông số điện năng từ các đồng hồ đo công suất đa năng, cụ thể là dòng thiết bị Power Meter. Quá trình nghiên cứu tập trung vào việc nghiên cứu sử dụng các chuẩn giao thức phổ biến hiện nay trong công nghiệp để thu thập các thông số điện năng quan trọng như điện áp, dòng điện, công suất thực, công suất biểu kiến, năng lượng tiêu thụ,..., ngoài ra còn tập trung nghiên cứu cách thức đưa dữ liệu thu thập được lên các hệ thống quản lý từ xa thông qua giao thức MQTT và giao thức HTTP. Bên cạnh đó, đề tài cũng hướng đến việc tối ưu hóa giao diện người dùng cục bộ thông qua màn hình LCD và người dùng có thể tương tác với thiết bị thông qua các nút nhấn, giúp kỹ thuật viên có thể tương tác và cấu hình hệ thống trực tiếp mà không cần thiết bị ngoại vi phức tạp.

1.2 Tình hình nghiên cứu trong và ngoài nước

Trên thế giới, việc nghiên cứu các bộ chuyển đổi giao thức (Gateway) cho mạng truyền thông công nghiệp đã trải qua nhiều giai đoạn phát triển. Các nghiên cứu ban đầu tập trung vào việc tối ưu hóa tính toán cho các dòng vi điều khiển 8-bit hoặc 16-bit để thực hiện các ngăn xếp giao thức phức tạp. Hiện nay, xu hướng quốc tế đang tập trung vào các hệ thống nhúng hiệu năng cao có tích hợp kết nối không dây. Các tập đoàn lớn như Moxa, Advantech hay Schneider Electric đã cung cấp các bộ Gateway công nghiệp (ví dụ dòng MGate) với độ tin cậy cực cao, hỗ trợ đa giao thức từ Modbus RTU sang Modbus TCP/IP hoặc MQTT. Tuy nhiên, các thiết bị này thường có giá thành rất cao và tính tùy biến về giao diện người dùng (HMI) cục bộ còn hạn chế.

Tại Việt Nam, lĩnh vực thiết kế thiết bị IoT và Gateway công nghiệp đang phát triển mạnh mẽ, đặc biệt là trong bối cảnh các nhà máy đang thực hiện chuyển đổi số. Nhiều nghiên cứu tập trung vào việc sử dụng các nền tảng vi điều khiển phổ biến như ESP32, STM32 để xây dựng các giải pháp Gateway tùy chỉnh với chi phí thấp. Nhiều doanh nghiệp trong nước đã bắt đầu nội địa hóa các bộ thu thập dữ liệu (Data Logger) cho hệ thống điện mặt trời và nhà máy sản xuất. Các nghiên cứu này thường tập trung vào việc tích hợp các giao thức truyền thông công nghiệp như Modbus RTU, Modbus TCP/IP, MQTT để thu thập dữ liệu từ các thiết bị đo lường và truyền tải lên các hệ thống quản lý từ xa. Tuy nhiên, vẫn còn nhiều thách thức về độ ổn định của kết nối không dây, bảo mật dữ liệu và tối ưu hóa giao diện người dùng cục bộ.

1.3 Nhiệm vụ đề tài

Nội dung 1: Tìm hiểu lý thuyết liên quan đến giao thức Modbus, giao thức Ethernet, vi điều khiển ESP32.

Nội dung 2: Tìm hiểu lý thuyết về cách sử dụng một trang Web để hiển thị các thông số theo dõi sau khi thu thập từ các thiết bị.

Nội dung 3: Thiết kế phần cứng cho Gateway sử dụng vi điều khiển ESP32.

Nội dung 4: Phát triển phần mềm nhúng cho Gateway để thu thập dữ liệu từ các thiết bị đo lường thông qua giao thức Modbus RTU và truyền tải dữ liệu lên hệ thống quản lý từ xa thông qua giao thức MQTT và HTTP.

Nội dung 5: Thiết kế và phát triển trang Web để hiển thị các thông số theo dõi sau khi thu thập từ các thiết bị.

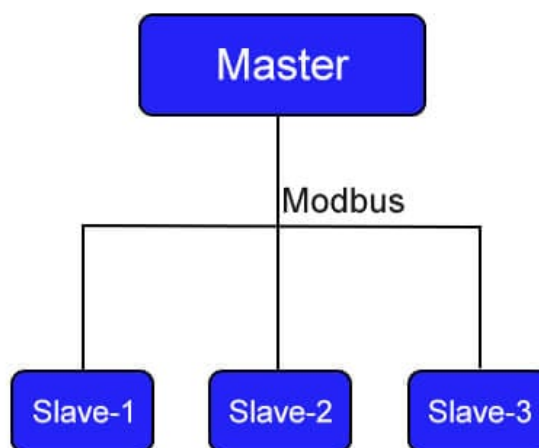
Nội dung 6: Thử nghiệm và đánh giá hiệu suất của hệ thống Gateway trong việc thu thập và truyền tải dữ liệu.

CHƯƠNG 2

Cơ sở lý thuyết

2.1 Tìm hiểu về lý thuyết về giao thức Modbus

2.1.1 Khái quát về giao thức Modbus



Hình 2.1: Tổng quan về giao thức Modbus

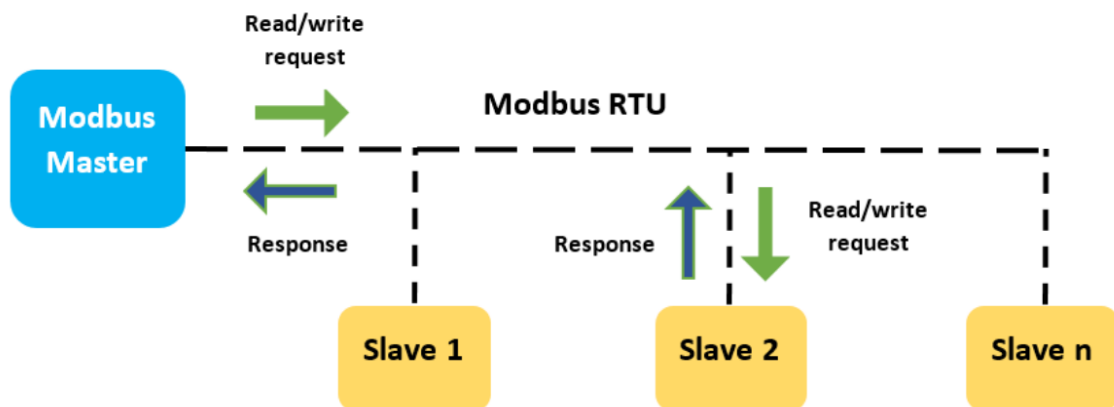
Giao thức Modbus là một chuẩn giao thức truyền thông công nghiệp được phát triển bởi Modicon vào năm 1979. Là một trong những giao thức truyền nhận đơn giản đáng tin cậy, có khả năng tương thích cao trong các hệ thống điện. Modbus được sử dụng chủ yếu để trao đổi dữ liệu giữa các thiết bị điện tử công nghiệp với nhau, hoạt động theo mô hình Client-Server (Master-Slaves), nghĩa là một Master giao tiếp với nhiều Slaves với nhau, có thể hiểu một cách đơn giản hơn là một thiết bị máy chủ có thể đọc được dữ liệu từ nhiều thiết bị con với nhau và các thiết bị con đó có thể truyền dữ liệu ngược lại cho thiết bị chủ miễn là chúng sử dụng chung một giao thức truyền nhận.

Giao thức được chia thành 3 biến thể thường được sử dụng trên thị trường, tùy thuộc vào môi trường làm việc sẽ sử dụng giao thức sao cho phù hợp, gồm 3 dạng chính sau:

- **Modbus RTU:** dữ liệu truyền dưới dạng số nhị phân, mô hình này sử dụng dạng truyền nhận có dây thường là các chuẩn như RS485, RS232 phù hợp với các mô hình giao tiếp nhỏ.
- **Modbus ASCII:** tương tự với Modbus RTU nhưng dữ liệu được truyền đi dưới dạng mã ASCII, có tốc độ chậm hơn Modbus RTU.
- **Modbus TCP/IP:** là hình thức truyền dữ liệu không dây theo chuẩn giao thức TCP/IP, sử dụng mạng Ethernet để truyền và nhận dữ liệu, thông dụng hơn trong các hệ thống hiện đại.

2.1.2 Tìm hiểu lý thuyết về giao thức Modbus RTU

2.1.2.1 Tổng quan về giao thức Modbus RTU



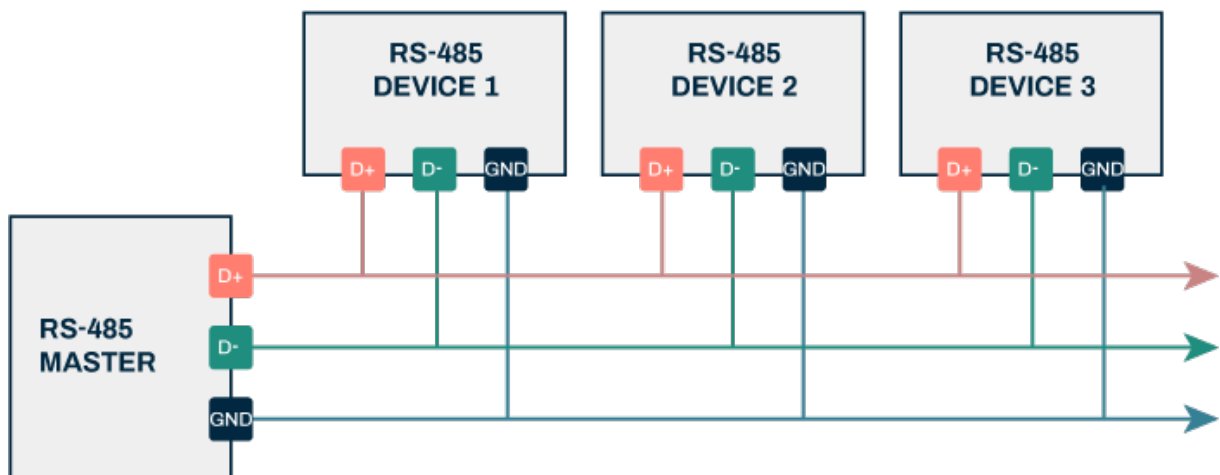
Hình 2.2: Tổng quan về giao thức Modbus RTU

Giao thức Modbus RTU (Remote Terminal Unit) là một biến thể của Modbus, là một giao thức truyền thông nối tiếp dựa trên mô hình Master-Slave, được sử dụng rộng rãi trong các hệ thống tự động hóa nhờ tính đơn giản và ổn định. Giao thức này hoạt động dựa trên việc mã hóa dữ liệu dưới dạng nhị phân (Binary), giúp tối ưu hóa tốc độ truyền tải và hiệu suất băng thông.

Chạy trên nền tảng vật lý là RS485 (cho phép kết nối đa điểm với khoảng cách xa) hoặc RS232. Một thiết bị Master có thể kết nối và truy vấn tối đa lên đến 247 thiết bị Slave (mỗi Slave có một địa chỉ ID duy nhất từ 1 đến 247). Master sẽ gửi một yêu cầu (Request) chứa địa chỉ Slave, mã hàm (Function Code), địa chỉ thanh ghi và dữ liệu cần xử lý. Slave tương ứng sẽ nhận lệnh, thực hiện và phản hồi (Response) lại kết quả hoặc báo lỗi. Điểm đặc trưng của Modbus RTU là sử dụng

dùng phương pháp kiểm tra lỗi CRC (Cyclic Redundancy Check) 16-bit ở cuối mỗi khung truyền. Điều này đảm bảo dữ liệu không bị sai lệch do nhiễu điện từ trong môi trường công nghiệp.

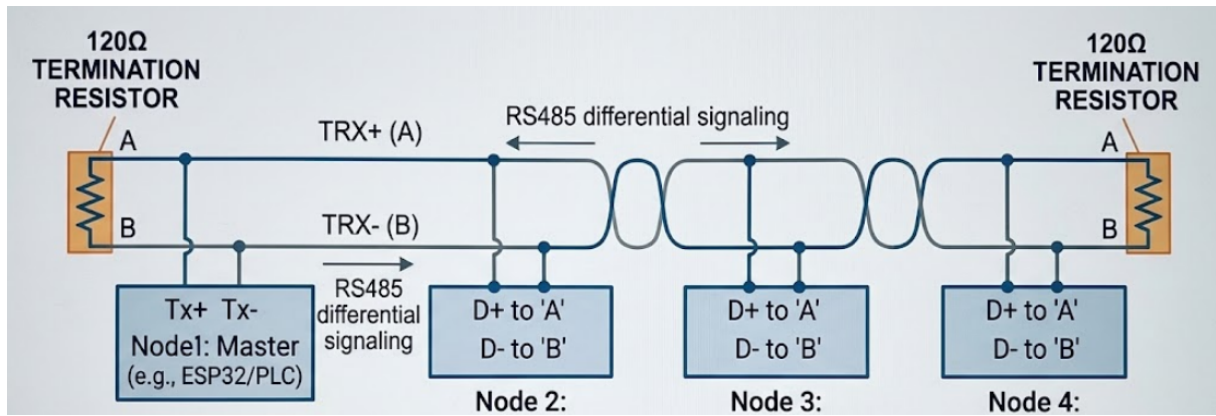
2.1.2.2 Tìm hiểu chuẩn vật lý RS485



Hình 2.3: Tổng quan về chuẩn vật lý RS485

RS485 (hay TIA/EIA-485) là chuẩn giao tiếp nối tiếp phổ biến nhất trong công nghiệp nhờ khả năng chống nhiễu cực tốt và truyền khoảng cách xa. Khác với RS232 sử dụng điện áp so với đất (Ground), RS485 sử dụng phương thức truyền tín hiệu vi sai (Differential Signaling) trên hai dây tín hiệu thường được ký hiệu là A (+) và B (-). RS485 hỗ trợ cấu trúc mạng Multi-drop, cho phép kết nối tối đa 32 thiết bị trên cùng một đường dây (với các chip hiện đại có thể lên đến 128 hoặc 256 thiết bị) trên khoảng cách tối đa 1200 mét.

Tín hiệu dữ liệu được xác định bằng sự chênh lệch điện áp giữa dây A và dây B. Khi có nhiễu điện từ tác động vào đường dây, nó thường ảnh hưởng lên cả hai dây cùng lúc. Nhờ bộ thu vi sai, thiết bị sẽ triệt tiêu nhiễu này, giúp tín hiệu vẫn giữ được độ chính xác ngay cả trong môi trường có nhiều motor hoặc biến tần.

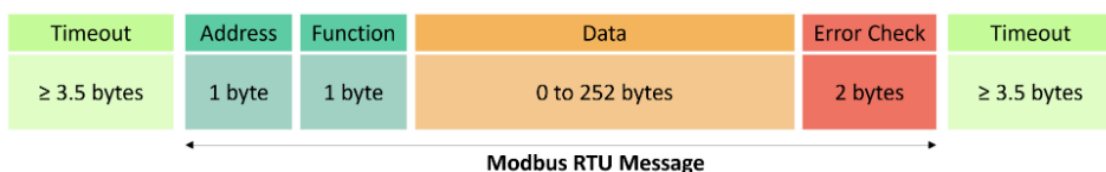


Hình 2.4: Sơ đồ kết nối RS485 với điện trở đầu cuối 120Ω

Trong mạng RS485, việc lắp đặt điện trở đầu cuối (thường có giá trị 120Ω) tại hai điểm đầu và cuối của đường bus là cực kỳ quan trọng. Điện trở này có nhiệm vụ phối hợp trở kháng của đường truyền. Khi tín hiệu điện di chuyển đến cuối đường dây, nếu không có điện trở này để "hấp thụ", tín hiệu sẽ bị phản xạ ngược lại (Signal Reflection).

Giúp Khử phản xạ tín hiệu, tránh việc các xung điện phản xạ đè lên xung điện mới, gây ra lỗi dữ liệu hoặc làm sai lệch khung truyền Modbus. Đảm bảo dạng sóng vuông của tín hiệu số được duy trì rõ ràng, đặc biệt quan trọng khi truyền ở tốc độ Baudrate cao hoặc dây dẫn quá dài. Giúp xác định mức điện áp rõ ràng khi đường bus đang ở trạng thái rảnh (Idle), tránh việc nhiễu làm thiết bị hiểu nhầm là có dữ liệu đang truyền. Ví dụ ta có +12V (điện áp trên dây A) và -12V (điện áp trên dây B), giá trị hiệu điện thế sẽ là 24V. Khi cặp dây A, B này được xoắn lại với độ dài bằng nhau, khi đó nguồn nhiễu từ môi trường tác động vào thì sẽ tác động đồng thời lên cả hai dây cùng một lúc (biên độ và pha của nguồn nhiễu ở hai dây gần như bằng nhau tại một thời điểm) thì khi đó nhiễu trên dây A và dây B tín hiệu điện thế sẽ bằng 0V và triệt tiêu hoặc làm suy hao tín hiệu nhiễu, nhiễu có thể đến từ nhiều nguồn khác nhau có thể thể là nhiễu nhiệt (Thermal Noise), nhiễu điện từ (EMI), nhiễu xuyên kênh (Cross Talk).

2.1.2.3 Tìm hiểu khung truyền dữ liệu Modbus RTU



Hình 2.5: Khung truyền tổng quát của Modbus RTU

Một khung truyền Modbus RTU tiêu chuẩn bao gồm 4 thành phần chính được truyền dưới dạng các byte nhị phân liên tiếp. Tổng độ dài của một khung

hình không được vượt quá 256 byte. Cấu trúc chi tiết như sau:

- **Địa chỉ thiết bị (Slave Address - 1 byte):** Xác định thiết bị đích nhận lệnh (từ 1 đến 247). Địa chỉ 0 được dùng cho chế độ Broadcast (gửi cho tất cả các thiết bị).
- **Mã hàm (Function Code- 1 byte):** Cho biết yêu cầu mà Master muốn Slave thực hiện (ví dụ: 03 để đọc thanh ghi, 06 để ghi một thanh ghi đơn).
- **Dữ liệu (Data - 0-252 byte):** Chứa thông tin bổ sung tùy theo mã hàm, bao gồm địa chỉ thanh ghi, số lượng thanh ghi cần đọc hoặc giá trị cần ghi vào thiết bị.
- **Kiểm tra lỗi (CRC - 2 byte):** Sử dụng thuật toán kiểm tra dư thừa vòng (Cyclic Redundancy Check) 16-bit để đảm bảo tính toàn vẹn của dữ liệu trong quá trình truyền dẫn.

Quy định về thời gian (Timing): Trong chuẩn RTU, các byte trong một khung phải được truyền liên tục. Một khung truyền được xác định là kết thúc khi có một khoảng lặng (silence) trên đường truyền tối thiểu bằng thời gian truyền 3.5 ký tự (3.5 character time). Nếu khoảng cách giữa hai byte vượt quá 1.5 ký tự, thiết bị nhận sẽ coi đó là một lỗi và hủy bỏ khung truyền đó.

Slave Address (1 Byte)	Function Code (1 Byte)	Start Address (2 Bytes)	Quantity (2 Bytes)	CRC (2 Bytes - CRC bit)
---------------------------	---------------------------	----------------------------	-----------------------	----------------------------

Hình 2.6: Khung truyền Master gửi Request cho Slave trong Modbus RTU

Khung truyền yêu cầu (Master Request): được khởi xướng bởi thiết bị Master để yêu cầu dữ liệu hoặc ra lệnh cho một Slave cụ thể. Ngoài Slave Address và Function Code, khung truyền này chứa thông tin định hướng gồm Start Address (địa chỉ thanh ghi bắt đầu) và Quantity (số lượng thanh ghi cần thao tác). Cả hai trường này đều có độ dài 2 Byte, cho phép Master truy cập chính xác vào các vùng nhớ dữ liệu bên trong thiết bị Slave. Kết thúc khung truyền là mã CRC 16-bit để đảm bảo lệnh gửi đi không bị nhiễu làm sai lệch.

Slave Address (1 Byte)	Function Code (1 Byte)	Data (N Bytes)	CRC (2 Bytes - CRC bit)
---------------------------	---------------------------	-------------------	----------------------------

Hình 2.7: Khung truyền Slave gửi Response cho Master trong Modbus RTU

Khung truyền phản hồi bình thường (Slave Response): sau khi nhận được lệnh hợp lệ, thiết bị Slave sẽ gửi lại khung truyền Response để xác nhận hoặc trả về dữ liệu. Cấu trúc khung này lặp lại Slave Address và Function Code từ yêu cầu của Master để xác nhận đối tượng đang giao tiếp. Phần quan trọng nhất là trường Data (có độ dài N Bytes tùy thuộc vào lượng dữ liệu yêu cầu), chứa các giá trị thực tế từ thanh ghi hoặc trạng thái của Slave. Khung truyền này giúp Master cập nhật được thông tin từ các cảm biến hoặc thiết bị ngoại vi trong hệ thống.

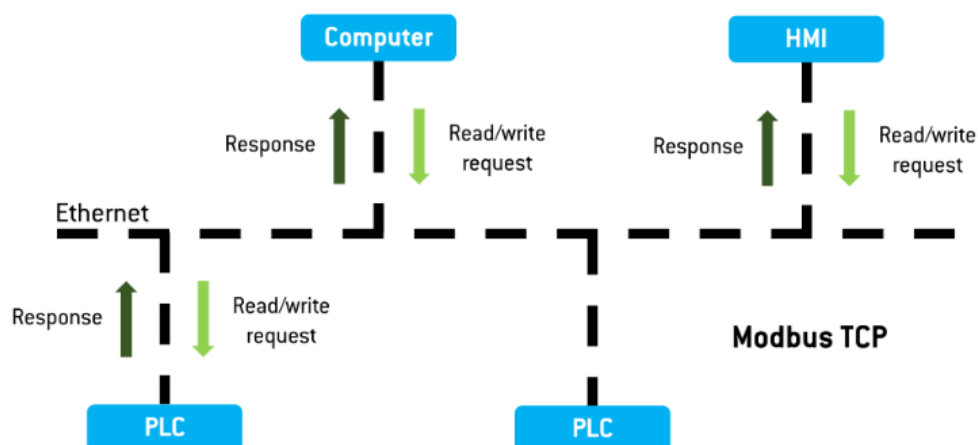
Slave Address (1 Byte)	Error Function Code (1 Byte)	Exception Code (1 Byte)
---------------------------	---------------------------------	----------------------------

Hình 2.8: Khung truyền Slave gửi báo lỗi cho Master trong Modbus RTU

Khung truyền báo lỗi (Exception Response): trong trường hợp Slave nhận được yêu cầu nhưng không thể thực hiện (ví dụ: sai địa chỉ thanh ghi hoặc mã hàm không hỗ trợ), nó sẽ gửi một khung phản hồi ngoại lệ. Điểm khác biệt nằm ở Error Function Code, giá trị này bằng mã hàm gốc cộng thêm $0x80$ (set bit MSB lên 1) để Master nhận diện đây là một thông báo lỗi. Đi kèm là Exception Code (1 Byte) dùng để mã hóa nguyên nhân cụ thể dẫn đến lỗi (như Illegal Data Address hoặc Illegal Function), giúp người vận hành dễ dàng chẩn đoán sự cố trong mạng truyền thông.

2.1.3 Khái niệm về giao thức Modbus TCP/IP

2.1.3.1 Tổng quan về giao thức Modbus TCP/IP



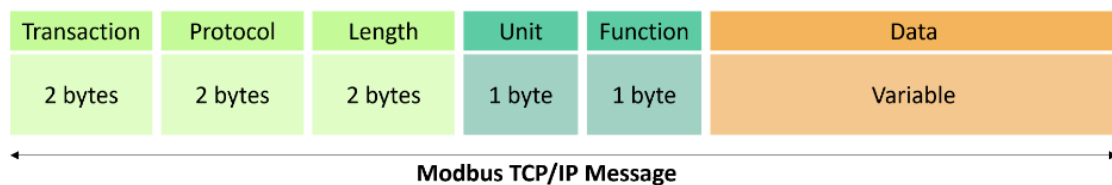
Hình 2.9: Tổng quan về giao thức Modbus TCP/IP

Giao thức Modbus TCP là một giao thức phần mềm hoạt động giống nhau trên cả mạng có dây và không dây, nhưng tốc độ, độ ổn định, độ trễ và độ tin

cây sẽ khác nhau tùy thuộc vào hạ tầng vật lý bên dưới (có dây hay không dây), khác với Modbus RTU thì Modbus TCP không cần tính toán CRC vì TCP đã có checksum và cơ chế đảm bảo toàn vẹn gói tin, tự động phát hiện/truyền lại nếu lỗi. Modbus TCP/IP sử dụng cổng mặc định 502, hoạt động trên nền tảng mạng Ethernet, Đảm bảo dữ liệu đầy đủ, đúng thứ tự, được ứng dụng vào Web (HTTP/HTTPS), Email, Modbus TCP. Trước khi truyền dữ liệu TCP thiết lập kết nối 3 bước (Three-way handshake) giữa 2 thiết bị (client và server), trong quá trình truyền diễn ra theo thứ tự như sau:

- Client gửi SYN (synchronize) đến Server để yêu cầu thiết lập kết nối.
- Server phản hồi bằng SYN-ACK (synchronize-acknowledge) để chấp nhận yêu cầu.
- Client gửi ACK (acknowledge) để xác nhận kết nối đã được thiết lập thành công.

2.1.3.2 Tìm hiểu về khung truyền dữ liệu Modbus TCP/IP



Hình 2.10: Khung truyền dữ liệu tổng quát của Modbus TCP/IP

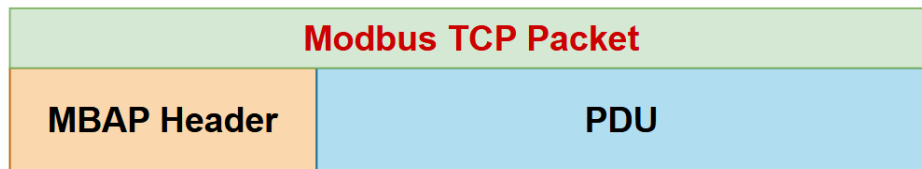
Một gói tin Modbus TCP/IP đầy đủ bao gồm phần tiêu đề MBAP Header và phần dữ liệu PDU (Protocol Data Unit). Các thành phần chi tiết bao gồm:

- **Transaction Identifier (2 byte):** Đây là mã định danh giao dịch dùng để phân biệt các giao dịch khác nhau, dùng để đồng bộ hóa giữa tin nhắn gửi đi và tin nhắn hồi đáp. Khi Master gửi một yêu cầu, nó sẽ gán một con số, và Slave khi phản hồi phải giữ nguyên con số này để Master biết phản hồi đó thuộc về yêu cầu nào.
- **Protocol Identifier (2 byte):** Luôn có giá trị 0x0000 cho Modbus TCP, dùng để xác định loại giao thức đang sử dụng.
- **Length (2 byte):** Trường độ dài. Nó cho biết số lượng byte còn lại trong gói tin, tính từ trường Unit Identifier cho đến hết trường Data. Điều này giúp thiết bị nhận biết được khi nào gói tin kết thúc.
- **Unit Identifier (1 byte):** Tương đương với Slave Address trong Modbus RTU. Trường này được dùng để định danh thiết bị Slave trong trường hợp

thiết bị đó nằm sau một Bridge hoặc Gateway (ví dụ một Modbus TCP Gateway chuyển đổi sang nhiều thiết bị Modbus RTU). Nếu giao tiếp trực tiếp qua IP, giá trị này thường được mặc định là 0xFF hoặc 0x01.

- **Function Code (1 byte):** Mã chức năng tương tự như Modbus RTU (ví dụ: 0x03 để đọc thanh ghi, 0x10 để ghi nhiều thanh ghi). Nó cho biết hành động mà Client yêu cầu Server thực hiện.
- **Data (N byte):** Chứa thông tin bổ sung tùy theo mã hàm, bao gồm địa chỉ thanh ghi, số lượng thanh ghi cần đọc hoặc giá trị cần ghi vào thiết bị.

Vì chạy trên nền TCP/IP, tầng liên lạc phía dưới đã có cơ chế kiểm tra tính toàn vẹn dữ liệu cực tốt, nên Modbus TCP không cần thêm 2 byte CRC ở cuối như RTU. Trong Modbus TCP/IP, Master được gọi là Client và Slave được gọi là Server, giao thức này thường chạy qua cổng TCP 502.



Hình 2.11: Thành phần của một gói tin Modbus TCP/IP khi được gửi đi

Transaction ID (2 Bytes)	Protocol ID (2 Bytes)	Length (2 Bytes)	Unit ID (1 Byte)	Function Code (1 Byte)	Start Address (2 Bytes)	Quantity (2 Bytes)
-----------------------------	--------------------------	---------------------	---------------------	---------------------------	----------------------------	-----------------------

Hình 2.12: Khung truyền yêu cầu Server gửi đến Client trong Modbus TCP/IP

Khung truyền yêu cầu (Request Message): là gói tin do Client (thường là máy tính hoặc Gateway) khởi tạo để gửi đến Server (thiết bị chấp hành). Điểm đặc trưng của khung này là phần dữ liệu PDU chứa hai thông tin quan trọng: Start Address (địa chỉ bắt đầu của thanh ghi) và Quantity (số lượng thanh ghi muốn truy cập). Các thông tin từ MBAP Header như Transaction ID giúp Client quản lý thứ tự các yêu cầu khi gửi nhiều lệnh liên tiếp trong môi trường mạng tốc độ cao.

Transaction ID (2 Bytes)	Protocol ID (2 Bytes)	Length (2 Bytes)	Unit ID (1 Byte)	Function Code (1 Byte)	Byte Count (2 Bytes)	Data (N Bytes)
-----------------------------	--------------------------	---------------------	---------------------	---------------------------	-------------------------	-------------------

Hình 2.13: Khung truyền phản hồi Client gửi đến Server trong Modbus TCP/IP

Khung truyền phản hồi (Response Message): khi Server nhận được yêu cầu hợp lệ, nó sẽ phản hồi bằng gói tin này. Điểm khác biệt lớn nhất so với gói tin Request là sự xuất hiện của trường Byte Count (Số lượng byte dữ liệu thực

tế sẽ gửi) và trường Data chứa giá trị của các thanh ghi. Độ dài (Length) trong MBAP Header cũng sẽ thay đổi tương ứng với kích thước của khối dữ liệu được trả về. Đây là gói tin hoàn tất một chu kỳ giao dịch (Transaction) giữa Client và Server.

Transaction ID (2 Bytes)	Protocol ID (2 Bytes)	Length (2 Bytes)	Unit ID (1 Byte)	Error Function Code (1 Byte)	Exception Code (1 Byte)
-----------------------------	--------------------------	---------------------	---------------------	---------------------------------	----------------------------

Hình 2.14: Khung truyền báo lỗi Client gửi đến Server trong Modbus TCP/IP

Khung truyền báo lỗi (Exception Response): trong trường hợp Server nhận được gói tin nhưng không thể thực hiện lệnh (ví dụ: địa chỉ thanh ghi không tồn tại hoặc dữ liệu không hợp lệ), nó sẽ gửi gói tin báo lỗi.

- **Error Function Code:** Có giá trị bằng mã hàm gốc cộng với 0x80 (ví dụ mã hàm 0x03 sẽ trở thành 0x83).
- **Exception Code:** Chỉ chứa duy nhất 1 Byte mã lỗi để thông báo chính xác nguyên nhân thất bại (như lỗi mã hàm không hỗ trợ hoặc địa chỉ không hợp lệ). Cấu trúc này giúp Client nhanh chóng nhận diện sự cố mà không cần phải chờ đợi timeout.

2.1.4 Tìm hiểu Function Codes trong giao thức Modbus

Function Codes	Description
0x01	Read Coils
0x02	Read Discrete Inputs
0x03	Read Holding Registers
0x04	Read Input Registers
0x05	Write Single Coils
0x06	Write Single Holding Register
0x0F	Write Multiple Coils
0x10	Write Multiple Holding Register

Bảng 2.1: Một số Function Codes phổ biến trong giao thức Modbus RTU

Function Code (mã lệnh) là thành phần quyết định loại thao tác dữ liệu mà Master yêu cầu Slave thực hiện, Được sử dụng cho cả Modbus RTU và Modbus TCP Các mã hàm phổ biến được phân chia dựa trên kiểu dữ liệu (Bit hoặc Word) và quyền truy cập (Đọc hoặc Ghi):

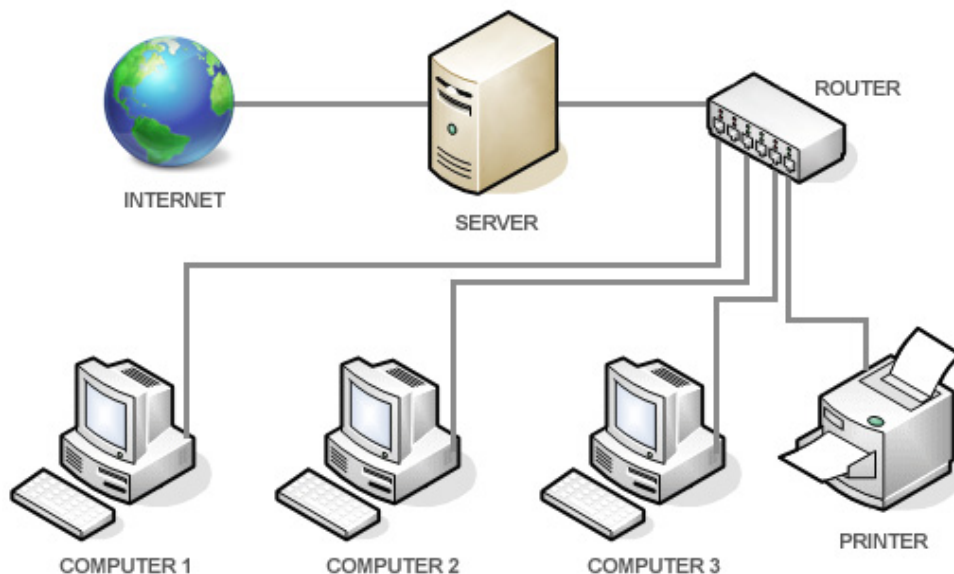
- **Nhóm đọc trạng thái Logic (0x01 và 0x02):** Mã hàm 0x01 (Read Coils) được sử dụng để đọc trạng thái ON/OFF của các đầu ra số như Relay hoặc đèn báo. Trong khi đó, mã hàm 0x02 (Read Discrete Inputs) dùng để kiểm tra trạng thái của các đầu vào số vật lý như nút nhấn hoặc công tắc hành trình, vốn là các dữ liệu chỉ cho phép đọc.

- **Nhóm đọc dữ liệu thanh ghi (0x03 và 0x04):** Để lấy dữ liệu 16-bit, mã hàm 0x03 (Read Holding Registers) thường được dùng để đọc các thông số cấu hình hoặc giá trị đo lường có khả năng thay đổi. Ngược lại, mã hàm 0x04 (Read Input Registers) chuyên dùng để đọc các giá trị từ cảm biến hoặc dữ liệu thô từ phần cứng mà Master không có quyền ghi đè.
- **Nhóm ghi dữ liệu đơn lẻ (0x05 và 0x06):** Khi cần điều khiển một đối tượng duy nhất, Master sử dụng mã hàm 0x05 (Write Single Coil) để đóng/ngắt một Relay hoặc mã hàm 0x06 (Write Single Register) để thay đổi giá trị của một thanh ghi cấu hình cụ thể (như cài đặt tần số cho biến tần).
- **Nhóm ghi dữ liệu đa nhiệm (0x0F và 0x10):** Để tối ưu hóa tốc độ truyền thông, mã hàm 0x0F (Write Multiple Coils) và 0x10 (Write Multiple Registers) cho phép Master ghi đồng thời một nhóm nhiều trạng thái Bit hoặc nhiều thanh ghi 16-bit liên tiếp chỉ trong một khung truyền duy nhất. Điều này đặc biệt hữu ích khi cần thiết lập đồng loạt nhiều tham số cho hệ thống.

2.2 Tìm hiểu về giao thức Ethernet

2.2.1 Khái quát và các đặc của giao thức Ethernet

Ethernet là giao thức truyền thông phổ biến nhất trong các mạng cục bộ (LAN), hoạt động chủ yếu ở tầng Vật lý (Physical) và tầng Liên kết dữ liệu (Data Link) trong mô hình 7 lớp OSI. Được chuẩn hóa theo chuẩn IEEE 802.3, Ethernet cho phép các thiết bị trong mạng chia sẻ dữ liệu thông qua các gói tin (frames) với tốc độ cao, từ 10 Mbps đến hàng trăm Gbps.

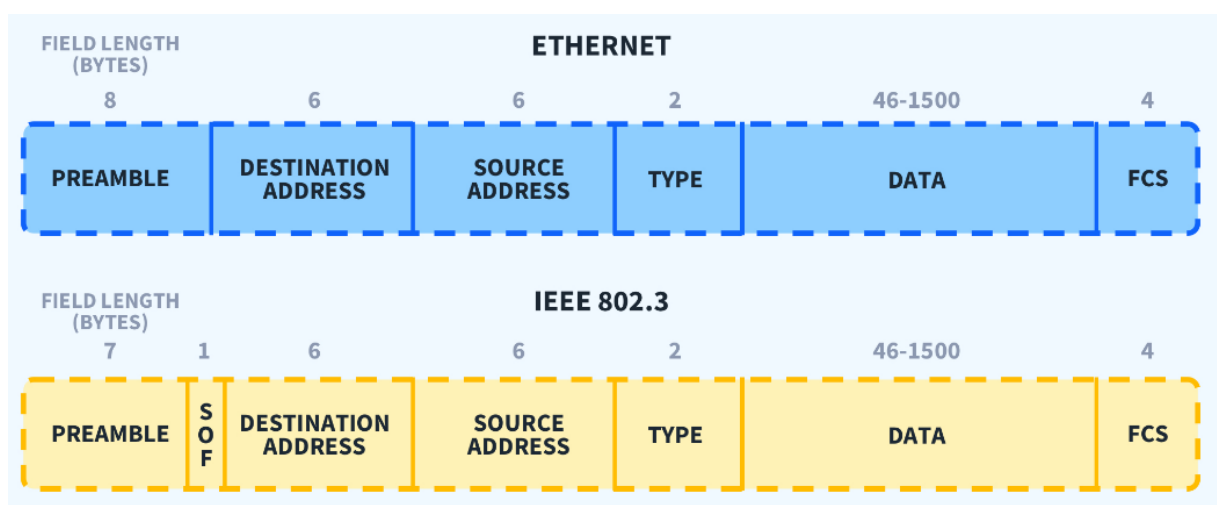


Hình 2.15: Tổng quan về mạng Ethernet

Các đặc trưng của giao thức Ethernet:

- **Cơ chế truy cập đường truyền (CSMA/CD):** Trước khi gửi dữ liệu, thiết bị sẽ "nghe" đường truyền. Nếu trống, nó mới gửi tin. Nếu có hai thiết bị gửi cùng lúc (xung đột), chúng sẽ dừng lại, chờ một khoảng thời gian ngẫu nhiên rồi mới thử lại. Trong các mạng hiện đại dùng Switch (Full-duplex), xung đột này hầu như đã được triệt tiêu.
- **Địa chỉ vật lý (MAC Address):** Mỗi thiết bị sử dụng mạng Ethernet để truyền dữ liệu đều có một địa chỉ MAC duy nhất gồm 48-bit (6 byte). Đây là định danh duy nhất để các thiết bị nhận diện nhau trong cùng một phân đoạn mạng.
- **Cấu trúc Frame:** Một gói tin Ethernet bao gồm các trường quan trọng như: Preamble (Đồng bộ hóa), Địa chỉ MAC nguồn/đích, Loại giao thức (Ether-Type), Dữ liệu (Payload) và mã kiểm tra lỗi FCS (Frame Check Sequence).
- **Chuẩn dây vật lý:** Sử dụng cáp xoắn đôi (Twisted Pair) như Cat5e, Cat6 với đầu nối RJ45 hoặc M12, giúp triệt tiêu nhiễu điện từ tương tự như nguyên lý vi sai của RS485 nhưng ở băng thông lớn hơn rất nhiều.
- **Tính đáp ứng thời gian thực:** Ethernet trong công nghiệp được cải tiến để chịu được môi trường khắc nghiệt (nhiều điện từ, nhiệt độ cao). Các giao thức như Modbus TCP/IP chạy trên nền Ethernet đảm bảo dữ liệu từ cảm biến được gửi về Gateway với độ trễ cực thấp.

2.2.2 Tìm hiểu về cấu trúc gói tin Ethernet



Hình 2.16: Cấu trúc của gói tin Ethernet

IEEE 802.3 là một tiêu chuẩn do tổ chức IEEE ban hành nhằm chuẩn hóa quy tắc chung cho công nghệ Ethernet, tập hợp các chuẩn của IEEE định nghĩa lớp vật lý và lớp con MAC của tầng liên kết dữ liệu trong mạng Ethernet có dây.

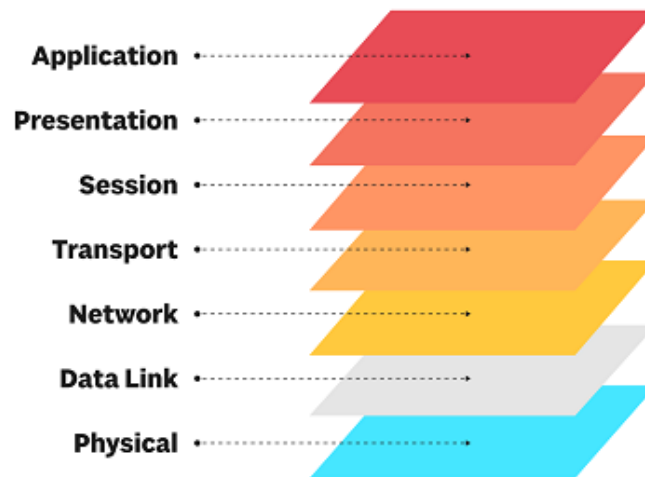
Các thành phần của một gói tin Ethernet như sau:

- **Preamble (Ethernet (8 bytes) / IEEE 802.3 (7 bytes))**: Là một dãy các bit 0 và 1 xen kẽ nhau. Có chức năng giúp thiết bị nhận đồng bộ hóa xung nhịp (clock) với thiết bị gửi.
- **SOF (Start of Frame Delimiter - 1 byte)**: Chỉ xuất hiện riêng biệt trong chuẩn IEEE 802.3, Ở Ethernet nó được tính gộp vào byte cuối của Preamble. có chức năng Đánh dấu điểm kết thúc của quá trình đồng bộ và báo hiệu rằng byte tiếp theo chính là địa chỉ đích.
- **Destination Address (6 bytes)**: chứa địa chỉ MAC đích, thiết bị nhận sẽ nhìn vào đây, nếu MAC này trùng với MAC của nó (hoặc là địa chỉ Broadcast), nó sẽ tiếp tục đọc gói tin nếu không nó sẽ bỏ qua gói tin.
- **Source Address (6 bytes)**: Chứa địa chỉ MAC nguồn (thiết bị gửi gói tin), giúp thiết bị nhận biết ai đã gửi dữ liệu để có thể phản hồi lại khi cần.
- **Type (2 bytes)**: Xác định giao thức tầng mạng (Layer 3) đang nằm bên trong phần Data, sử dụng IPv4 hay IPv6.
- **Data (Payload) (46 - 1500 bytes)**: Chứa dữ liệu người dùng muốn gửi đi.
- **FCS (Frame Check Sequence - 4 bytes)**: Sử dụng thuật toán kiểm tra mã CRC. Thiết bị nhận sẽ tính toán lại mã này từ dữ liệu nhận được và so sánh với mã FCS ở cuối gói tin. Nếu không khớp, gói tin bị coi là lỗi và bị hủy bỏ ngay lập tức.

2.2.3 Cách thức đóng gói gói tin Ethernet theo mô hình OSI

2.2.3.1 Tìm hiểu về mô hình OSI

The OSI Model



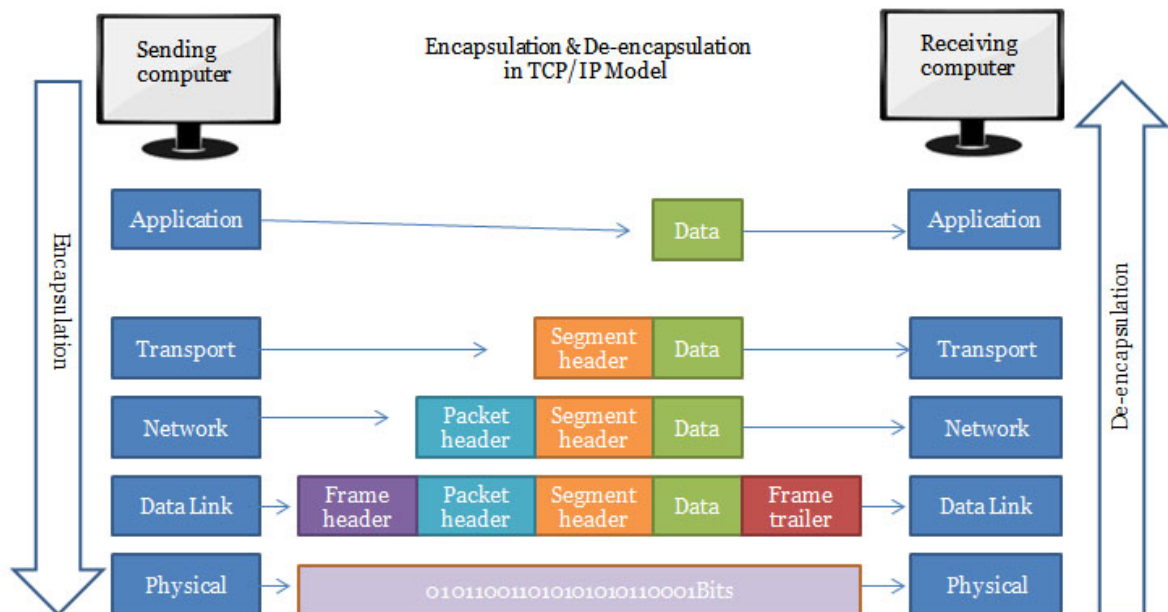
Hình 2.17: Cấu trúc tổng quát của mô hình OSI

Mô hình OSI (Open Systems Interconnection) là một khung lý thuyết được bộ tiêu chuẩn quốc tế (ISO) đưa ra để chuẩn hóa cách các hệ thống máy tính truyền thông với nhau Chức năng của từng thành phần trong mô hình:

- **Application Layer:** Giao diện trực tiếp với người dùng và các tiến trình phần mềm, nơi data chính trong gói tin được sinh ra, như HTTP (Web), MQTT (IoT), Modbus (Dữ liệu người dùng), Email (SMTP).
- **Presentation Layer:** lo việc định dạng dữ liệu, mã hóa (Encryption) và nén dữ liệu để đảm bảo máy nhận có thể đọc được.
- **Session Layer:** thiết lập, duy trì và đồng bộ hóa cuộc hội thoại giữa hai thiết bị. Nếu kết nối bị ngắt, tầng này lo việc khôi phục từ điểm kiểm soát.
- **Transport Layer:** Đảm bảo dữ liệu đi từ điểm A đến điểm B một cách toàn vẹn. Nó phân đoạn dữ liệu lớn thành các mảnh nhỏ và kiểm soát lưu lượng để tránh nghẽn mạng.
- **Network Layer:** Quyết định lộ trình tốt nhất để gói tin đi qua các mạng khác nhau dựa trên địa chỉ IP.

- **Data Link Layer:** Đóng gói dữ liệu thành các Frame, kiểm tra lỗi vật lý (CRC) và định địa chỉ vật lý (MAC).
- **Physical Layer:** Truyền dẫn dòng bit thô qua các phương tiện vật lý bằng tín hiệu điện, ánh sáng hoặc sóng vô tuyến.

2.2.3.2 Cách thức đóng gói tin Ethernet

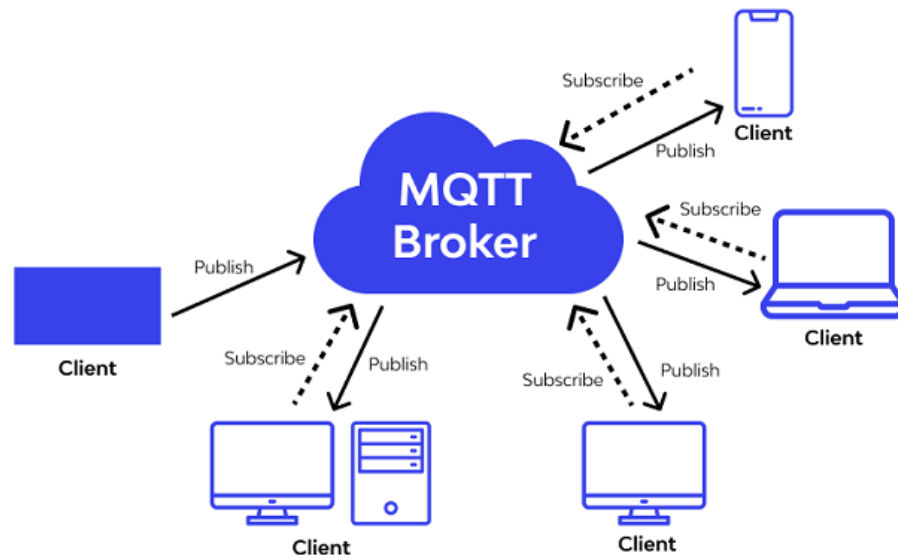


Hình 2.18: Quy trình đóng gói tin Ethernet

Các bước đóng gói gói tin Ethernet:

- **Application, Presentation, Session - Layer 7, 6, 5:** Chuyển đổi dữ liệu người dùng từ ứng dụng sang dạng nhị phân, mã hóa và thiết lập phiên truyền dẫn.
- **Transport - Layer 4:** Chia nhỏ dữ liệu (Segmentation) và thêm Transport Header, chịu trách nhiệm vận chuyển đầu-cuối. Thông tin quan trọng nhất ở đây là Số hiệu cổng (Port Number) để xác định ứng dụng nào đang gửi/nhận.
- **Network - Layer 3:** Bao bọc Segment bằng một Network Header. Thông tin cốt lõi là Địa chỉ Logic (như địa chỉ IP) của máy nguồn và máy đích.
- **Data Link - Layer 2:** Bao bọc Packet bằng một Data Link Header (chứa địa chỉ vật lý như MAC) và một Trailer (chứa mã kiểm tra lỗi FCS/CRC), chuẩn bị dữ liệu để truyền trên đường dây vật lý cụ thể.
- **Physical - Layer 1:** Chuyển toàn bộ Frame thành các xung điện, ánh sáng hoặc sóng vô tuyến.

2.3 Tìm hiểu về giao thức truyền thông MQTT



Hình 2.19: Mô hình giao tiếp MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông không dây, chạy trên nền tảng TCP/IP, có mã nguồn mở, được thiết kế cho các ứng dụng IoT với băng thông thấp, độ trễ cao, hoặc thiết bị có tài nguyên hạn chế. Là một phần trung gian cho phép các MQTT Client giao tiếp với nhau thông qua các chủ đề (Topic) đã đăng ký, có thể hiểu một cách đơn giản MQTT như một nhà phân phối sản phẩm theo nhu cầu của từng khách hàng thông qua Topic mà khách hàng đã đăng ký. Hoạt động dựa trên mô hình chính là Publish/Subscribe (Pus/Sub) và các thông tin được tổ chức theo Topic, các Slaves có thể gửi dữ liệu theo chủ đề để gửi đến những thiết bị Master (Subscriber) đã đăng ký nhận dữ liệu đó.

Vì là giao thức cần ít dung lượng nên MQTT được ứng dụng phổ biến trong các ứng dụng IoT sử dụng cảm biến thu thập các thông tin về môi trường, có thể kể đến một số ứng dụng sử dụng giao thức MQTT làm trung gian như sau:

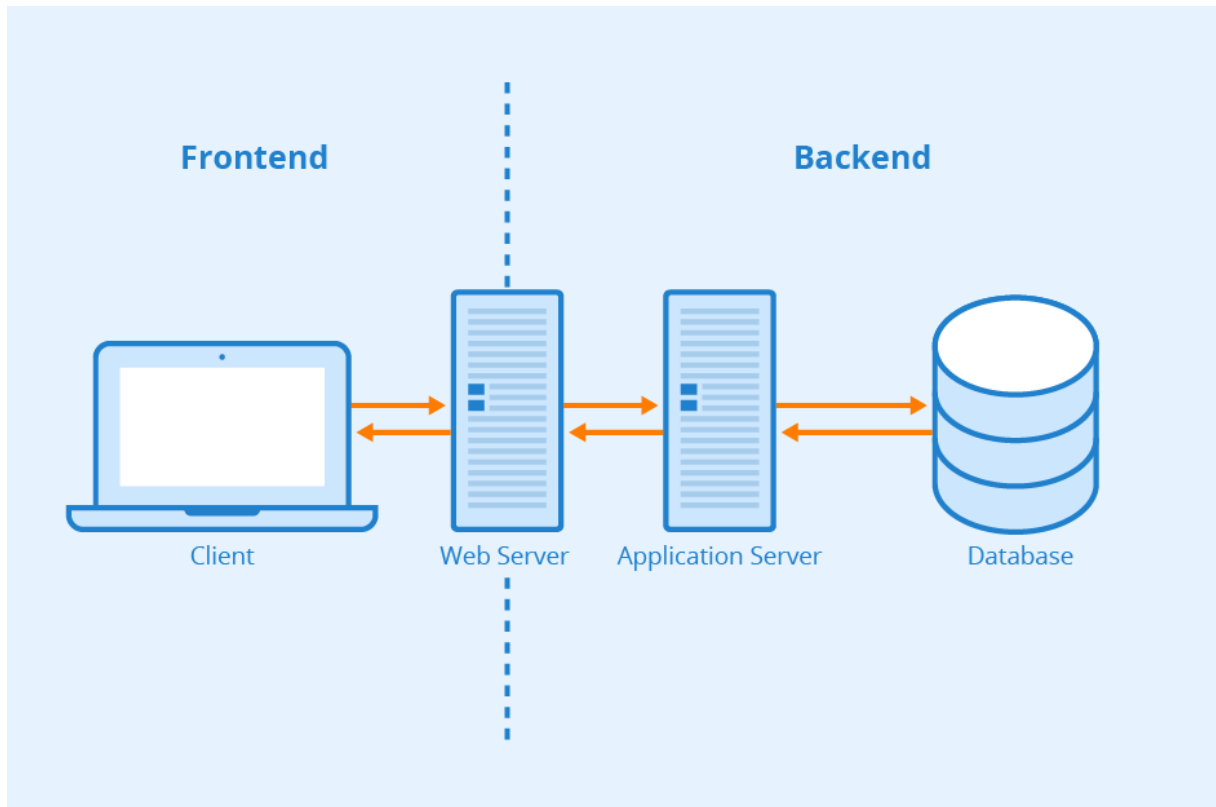
- Thu thập dữ liệu từ cảm biến nhiệt độ, độ ẩm và đưa lên Web theo dõi.
- Gửi lệnh điều khiển từ trên Web để điều khiển bật/tắt các Relay, máy bơm.
- Cấu hình các thông số của hệ thống IoT từ xa.
- Gửi cấu hình dữ liệu từ Web hoặc ứng dụng trên điện thoại tới nhiều thiết bị cùng một lúc.

Trong MQTT có hai phần chính là MQTT Broker và MQTT Clients như sau:

- **MQTT Broker:** là nơi tiếp nhận các thông tin cần xử lý của các thiết bị trong hệ thống, chịu trách nhiệm định tuyến hướng đi của các dữ liệu sao cho đến được chính xác vị trí mong muốn cần truyền giữa hai thiết bị đang muốn giao tiếp với nhau, có thể coi như là một nhà trung gian giữa hai thiết bị
- **MQTT Client:** là các thiết bị thu thập dữ liệu hay các thiết bị điều khiển hoặc cũng có thể là các ứng dụng, Web. Các thiết bị này sẽ không trực tiếp giao tiếp với nhau mà chúng sẽ thông qua MQTT Broker, hoạt động dựa trên thư viện MQTT và thiết lập kết nối với MQTT Broker.

2.4 Tìm hiểu về các kiến thức liên quan tới trang web

Một ứng dụng web hiện đại là hệ thống phần mềm hoạt động theo mô hình Client-Server, được cấu thành từ ba trụ cột chính: Frontend (Giao diện), Backend (Hệ thống xử lý) và Database (Cơ sở dữ liệu). Trong đó, Frontend chịu trách nhiệm hiển thị nội dung và tương tác trực quan với người dùng thông qua các công nghệ cốt lõi như HTML, CSS và JavaScript. Phía sau giao diện là Backend, nơi chứa các logic nghiệp vụ (Business Logic), thuật toán xử lý và các API để điều phối dữ liệu. Cuối cùng, Database đóng vai trò là kho lưu trữ thông tin bền vững, quản lý việc truy xuất và cập nhật dữ liệu. Các thành phần này được liên kết chặt chẽ và vận hành trên hạ tầng Internet thông qua giao thức mạng HTTP/HTTPS, hệ thống tên miền (DNS) và máy chủ web (Web Server).



Hình 2.20: Tổng quan về các thành phần của một trang web

Các thành phần cốt lõi bao gồm:

- **Frontend (Client-side):** Là phần người dùng nhìn thấy và tương tác. Nó bao gồm cấu trúc trang (HTML), định dạng hiển thị (CSS) và các hiệu ứng/xử lý động (JavaScript/Frameworks như React, Vue).
- **Backend (Server-side):** Là phần chạy ngầm trên máy chủ, xử lý các yêu cầu từ Frontend. Nó bao gồm ngôn ngữ lập trình server (như Python, Node.js, PHP) và Web Server (như Nginx, Apache).
- **Database (Cơ sở dữ liệu):** Nơi lưu trữ dữ liệu người dùng, bài viết, sản phẩm... (thường là MySQL, PostgreSQL hoặc MongoDB).
- **Giao thức và Hạ tầng:** Bao gồm HTTP/HTTPS (luật truyền tải dữ liệu) và DNS (hệ thống phân giải tên miền giúp biến google.com thành địa chỉ IP máy chủ).

2.5 Tìm hiểu về Database Postgres

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ nhất hiện nay, nổi tiếng với độ tin cậy, tính toàn vẹn dữ liệu và tính năng phong phú. Hệ thống hỗ trợ các truy vấn phức tạp, khóa ngoại, trigger, view, và transactional integrity (ACID), đóng vai trò là nền tảng lưu trữ dữ liệu an toàn và

hiệu quả cho toàn bộ ứng dụng.

Đặc điểm: Đặc điểm nổi bật là kiến trúc Object-Relational (ORDBMS), cho phép kết hợp chặt chẽ giữa mô hình dữ liệu quan hệ truyền thống với các tính năng hướng đối tượng linh hoạt. Hệ thống này tuân thủ nghiêm ngặt chuẩn ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo tính toàn vẹn và an toàn tuyệt đối cho dữ liệu ngay cả trong các trường hợp sự cố hệ thống. Khác biệt với các hệ quản trị khác, PostgreSQL hỗ trợ mạnh mẽ các kiểu dữ liệu phi cấu trúc như JSONB và XML, cho phép nó hoạt động hiệu quả như một cơ sở dữ liệu lai (Hybrid) giữa SQL và NoSQL. Ngoài ra, cơ chế quản lý đồng thời đa phiên bản (MVCC - Multi-Version Concurrency Control) giúp tối ưu hóa hiệu suất xử lý, cho phép nhiều người dùng đọc và ghi dữ liệu cùng lúc mà không xảy ra tình trạng khóa (locking) gây tắc nghẽn hệ thống.



Hình 2.21: Hệ quản trị cơ sở dữ liệu Postgres

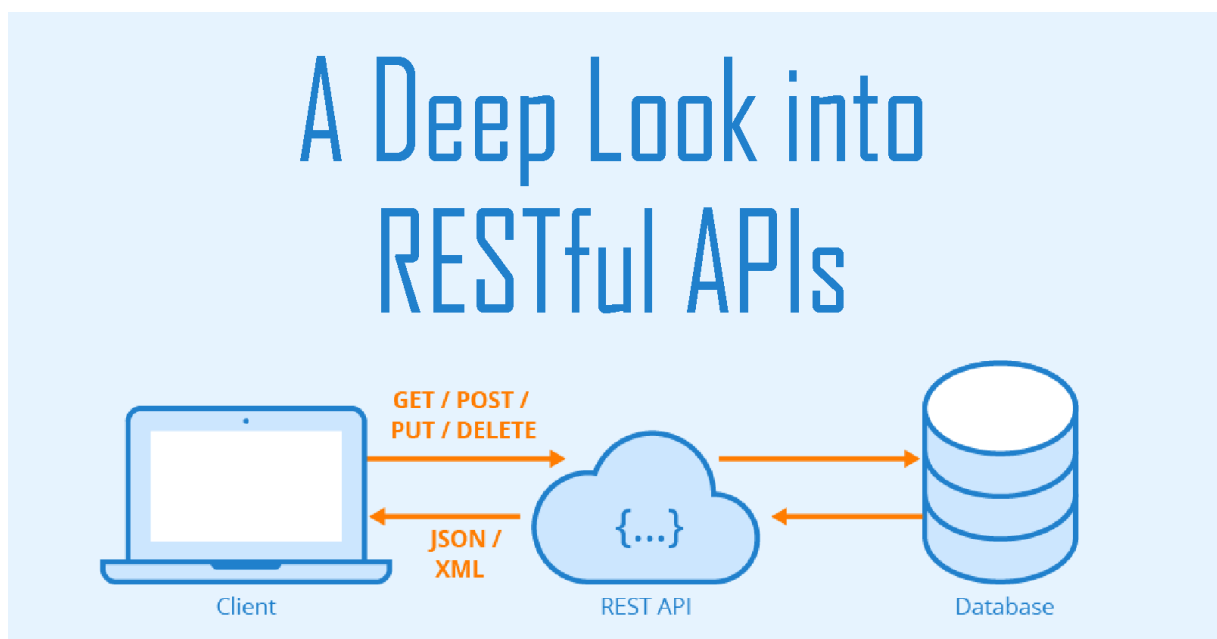
Khác biệt chính với MySQL: hệ quản trị cơ sở dữ liệu phổ biến nhất cho web, PostgreSQL thể hiện ưu thế vượt trội trong các bài toán đòi hỏi độ phức tạp cao và tính toàn vẹn dữ liệu nghiêm ngặt. Trong khi MySQL tập trung tối ưu tốc độ cho các tác vụ đọc (Read-heavy) đơn giản, PostgreSQL lại được thiết kế để xử lý hiệu quả các truy vấn phức tạp, các phép JOIN dữ liệu lớn và tuân thủ các tiêu chuẩn SQL chặt chẽ hơn. Đặc biệt, PostgreSQL vượt trội về khả năng hỗ trợ các kiểu dữ liệu hiện đại như JSONB và dữ liệu địa lý (Geospatial), iến nó thành lựa chọn tối ưu cho các hệ thống Enterprise cần sự linh hoạt, khả năng mở rộng và độ tin cậy cao thay vì chỉ tập trung vào tốc độ thuần túy.

2.6 Khái niệm về API RESTful và API Websocket

2.6.1 Khái niệm và đặc điểm về API RESTful

RESTful API (Representational State Transfer) là một chuẩn kiến trúc phần mềm dùng để thiết kế các dịch vụ Web (Web Services), cho phép các ứng dụng trao đổi dữ liệu qua lại thông qua giao thức HTTP. Trong mô hình này, dữ liệu được coi là các "Tài nguyên" (Resources) và được định danh bằng các đường dẫn URI (Uniform Resource Identifier). Nguyên tắc cốt lõi của RESTful API là sự đơn giản, độc lập (Stateless) và sử dụng các phương thức HTTP tiêu chuẩn để thực hiện các hành vi thao tác dữ liệu.

Việc áp dụng kiến trúc RESTful API giúp hệ thống tách biệt hoàn toàn giữa Frontend và Backend, cho phép phát triển độc lập và dễ dàng mở rộng. Nhờ sử dụng giao thức HTTP và định dạng JSON nhẹ nhàng, RESTful API tối ưu hóa hiệu suất truyền tải mạng, đồng thời giúp ứng dụng dễ dàng tích hợp với các nền tảng khác nhau (Web, Mobile App, IoT Devices) mà không cần thay đổi cấu trúc dữ liệu cốt lõi.



Hình 2.22: Tổng quan về RESTful API

Để một API được gọi là chuẩn RESTful, nó phải tuân thủ các nguyên tắc sau:

- **Stateless (Phi trạng thái):** Đây là đặc điểm quan trọng nhất. Server không lưu giữ trạng thái của Client giữa các lần gửi request. Mỗi request gửi lên phải chứa đầy đủ thông tin cần thiết (như token xác thực) để Server hiểu và xử lý.

Phương thức HTTP	Hành động tương ứng	Ý nghĩa	Ví dụ
GET	Read	Lấy dữ liệu từ server về.	GET /users (Lấy danh sách người dùng)
POST	Create	Gửi dữ liệu mới lên để tạo tài nguyên.	POST /users (Tạo người dùng mới)
PUT / PATCH	Update	Cập nhật thông tin của tài nguyên.	PUT /users/1 (Sửa thông tin user số 1)
DELETE	Delete	Xóa tài nguyên khỏi hệ thống.	DELETE /users/1 (Xóa user số 1)

Bảng 2.2: Các phương thức HTTP và hành động tương ứng

- **Resource-based (Dựa trên tài nguyên):** Mọi thứ đều được xem là tài nguyên và được truy cập qua một đường dẫn (Endpoint) cụ thể. Ví dụ: /products, /orders.
- **Representation (Định dạng dữ liệu):** Dữ liệu trả về cho Client thường ở định dạng JSON (phổ biến nhất hiện nay vì nhẹ và dễ đọc) hoặc XML.

HTTP Status Codes (Mã phản hồi) sử dụng chuẩn code để báo kết quả, bao gồm một số mã sử dụng chủ yếu như sau:

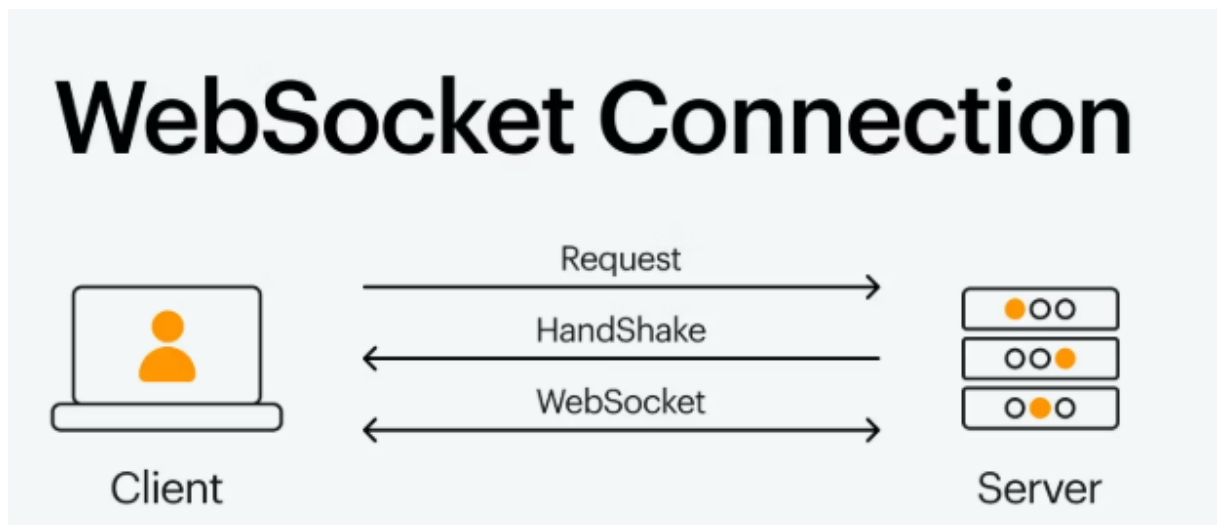
- **200 OK:** Thành công.
- **201 Created:** Tạo mới thành công.
- **400 Bad Request:** Lỗi do gửi dữ liệu sai.
- **401 Unauthorized:** Chưa đăng nhập/không có quyền.
- **404 Not Found:** Không tìm thấy tài nguyên.
- **500 Internal Server Error:** Lỗi tại Server.

2.6.2 Khái niệm và đặc điểm về API WebSocket

WebSocket là một giao thức truyền thông máy tính tiên tiến, cung cấp các kênh giao tiếp song phương (full-duplex) qua một kết nối TCP duy nhất. Khác với giao thức HTTP truyền thống (vốn hoạt động theo cơ chế 'hỏi - đáp' và ngắt kết nối ngay sau khi hoàn tất), WebSocket cho phép thiết lập một kết nối bền vững (persistent connection) giữa Client và Server. Nhờ đó, dữ liệu có thể được truyền tải qua lại tức thì trong thời gian thực (Real-time) mà không cần Client phải gửi

yêu cầu liên tục.

WebSocket là giải pháp xương sống cho các ứng dụng đòi hỏi tính tương tác cao và độ trễ thấp. Nếu RESTful API tối ưu cho việc quản lý tài nguyên (CRUD), thì WebSocket lại chiếm ưu thế tuyệt đối trong các ứng dụng thời gian thực (Real-time Applications). Các trường hợp sử dụng điển hình bao gồm: Ứng dụng chat trực tuyến, thông báo (Notification) thời gian thực, bảng giá chứng khoán/tiền ảo nhảy liên tục, game online nhiều người chơi, và các ứng dụng chỉnh sửa tài liệu cộng tác.



Hình 2.23: Tổng quan về Websocket API

WebSocket hoạt động khác biệt hoàn toàn so với mô hình Request-Response của REST, quá trình này diễn ra qua 2 giai đoạn chính sau:

- **Giai đoạn Bắt tay (Handshake):** ban đầu, Client gửi một HTTP Request thông thường nhưng kèm theo header Upgrade: websocket. Nếu Server chấp nhận, nó sẽ phản hồi mã 101 Switching Protocols. Lúc này, giao thức HTTP được chuyển đổi (upgrade) sang giao thức WebSocket.
- **Giai đoạn Truyền dữ liệu (Data Transfer):** kết nối TCP được giữ nguyên (không đóng lại). Dữ liệu được đóng gói thành các Frame nhẹ và truyền qua lại hai chiều bất cứ lúc nào. Server chủ động đẩy (Push) dữ liệu xuống Client ngay khi có tin mới mà không cần Client phải "hỏi".

Các đặc điểm liên quan tới Websocket API như sau:

- **Stateful (Có trạng thái):** Khác với REST (Stateless), trong WebSocket, Server nhận biết được kết nối của Client đang "sống" (alive). Nếu mất mạng, kết nối sẽ bị ngắt (Close) và cả hai bên đều biết.
- **Full-Duplex (Song công):** Dữ liệu có thể đi từ Client -> Server và Server -> Client cùng một lúc, độc lập với nhau.

- **Low Latency (Độ trễ cực thấp):** Do không phải gửi kèm các HTTP Header công kênh trong mỗi lần gửi tin (như REST), WebSocket giảm thiểu tối đa lưu lượng mạng, giúp tốc độ phản hồi cực nhanh.
- **Giao thức riêng:** Sử dụng ws:// (cho thường) và wss:// (cho bảo mật - tương đương HTTPS).
- **Sự kiện (Event-driven):** Lập trình WebSocket dựa trên các sự kiện: onOpen (khi nối thành công), onMessage (khi nhận tin), onClose (khi ngắt), onError (khi lỗi).

2.7 Tìm hiểu về phần mềm Postman

2.7.1 Khái niệm và Postman

Postman là một nền tảng cộng tác (Collaboration Platform) toàn diện dành cho việc phát triển và kiểm thử API (Application Programming Interface). Khởi đầu là một công cụ mở rộng của Chrome, Postman hiện đã phát triển thành một ứng dụng độc lập mạnh mẽ, cho phép lập trình viên gửi các yêu cầu HTTP (HTTP Requests), xem phản hồi từ máy chủ, gỡ lỗi (debug), viết kịch bản kiểm thử tự động (Automated Testing) và tạo tài liệu API một cách trực quan mà không cần viết dòng lệnh phức tạp.

Ngoài giao diện đồ họa, Postman còn cung cấp công cụ **Newman** chạy trên dòng lệnh, cho phép tích hợp các kịch bản kiểm thử API vào hệ thống CI/CD (như Jenkins, GitHub Actions) để tự động hóa quy trình kiểm soát chất lượng.



Hình 2.24: Logo của Postman

2.7.2 Các đặc điểm và vai trò chính của Postman

Các đặc điểm chính như sau:

- **API Client (Giao diện gửi yêu cầu):** Đây là tính năng cơ bản nhất. Nó cho phép người dùng chọn phương thức (GET, POST, PUT, DELETE...),

- **Testing:** Nhập URL, thêm Headers, Body (JSON, Form-data) và gửi đi để xem kết quả trả về (Status code, thời gian phản hồi, dữ liệu).
- **Collections (Bộ sưu tập):** Là nơi gom nhóm các API request lại với nhau theo từng dự án hoặc từng module chức năng (ví dụ: Folder "User Management", Folder "Payment"). Điều này giúp tổ chức và quản lý hàng trăm API một cách khoa học.
- **Environments (Môi trường):** Tính năng cho phép định nghĩa các biến môi trường (Variables) như base-url, token. Nhờ đó, lập trình viên có thể chuyển đổi linh hoạt giữa các môi trường (Development, Staging, Production) mà không cần sửa đổi thủ công từng đường dẫn API.
- **Mock Servers:** Cho phép tạo ra một máy chủ giả lập để Frontend có thể làm việc và gọi API ngay cả khi Backend chưa viết xong code thực tế.
- **Automated Testing (Kiểm thử tự động):** Postman cho phép viết các đoạn script (bằng JavaScript) để tự động kiểm tra xem kết quả trả về có đúng như mong đợi không (ví dụ: Kiểm tra Status code có phải là 200 không, user-id có tồn tại không).
- **Giao diện trực quan (GUI):** Dễ sử dụng cho cả người không chuyên sâu về code.
- **Hỗ trợ đa giao thức:** Không chỉ RESTful API, Postman hiện nay hỗ trợ cả SOAP, GraphQL, gRPC và WebSocket.
- **Cộng tác (Collaboration):** Cho phép chia sẻ Collection giữa các thành viên trong team, đồng bộ hóa lịch sử làm việc.
- **Tích hợp CI/CD:** Thông qua công cụ dòng lệnh Newman (phiên bản CLI của Postman), các bài test có thể được tích hợp vào quy trình triển khai phần mềm tự động.

Các vai trò chính trong quy trình phát triển CSDL như sau:

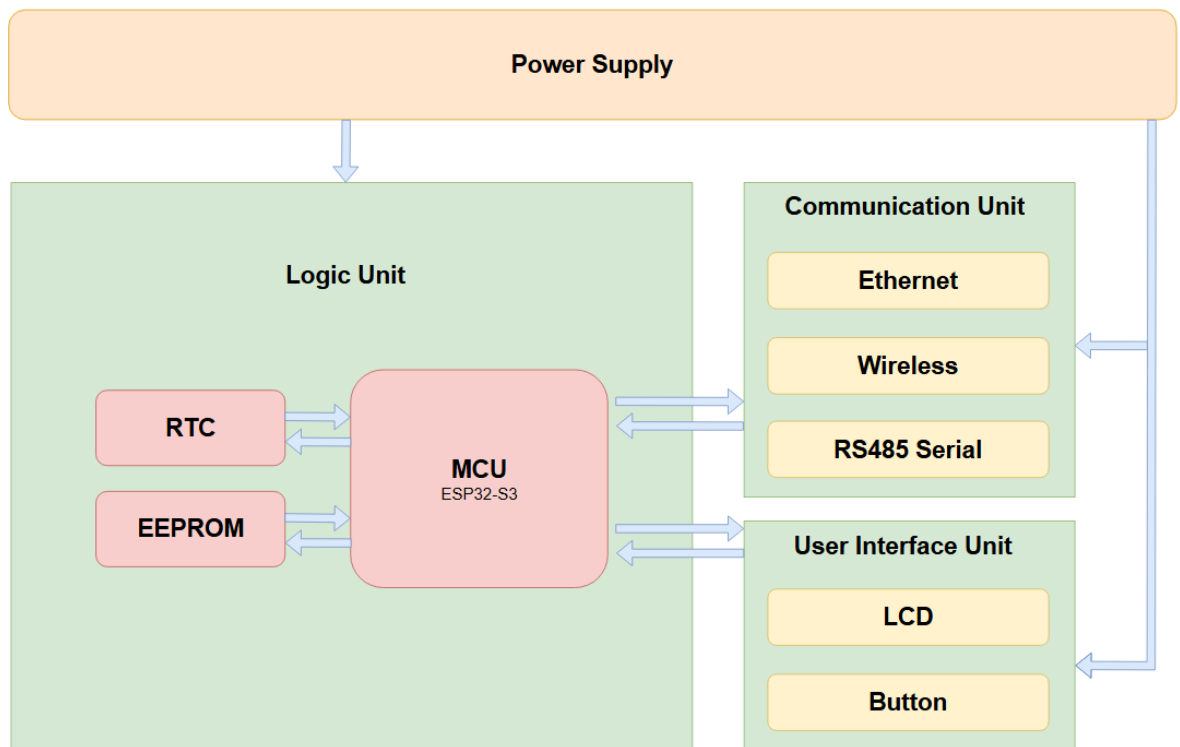
- **Đối với Backend Developer:** Dùng để kiểm tra nhanh API mình vừa viết xem đã chạy đúng logic chưa trước khi bàn giao.
- **Đối với Frontend Developer:** Dùng để hiểu cấu trúc dữ liệu JSON trả về, từ đó binding (gán) dữ liệu lên giao diện chính xác.
- **Đối với Tester/QA:** Dùng để thực hiện kiểm thử thủ công (Manual Test) hoặc chạy các bộ test tự động để đảm bảo hệ thống không bị lỗi (Regression Test).

CHƯƠNG 3

Kết quả thực hiện

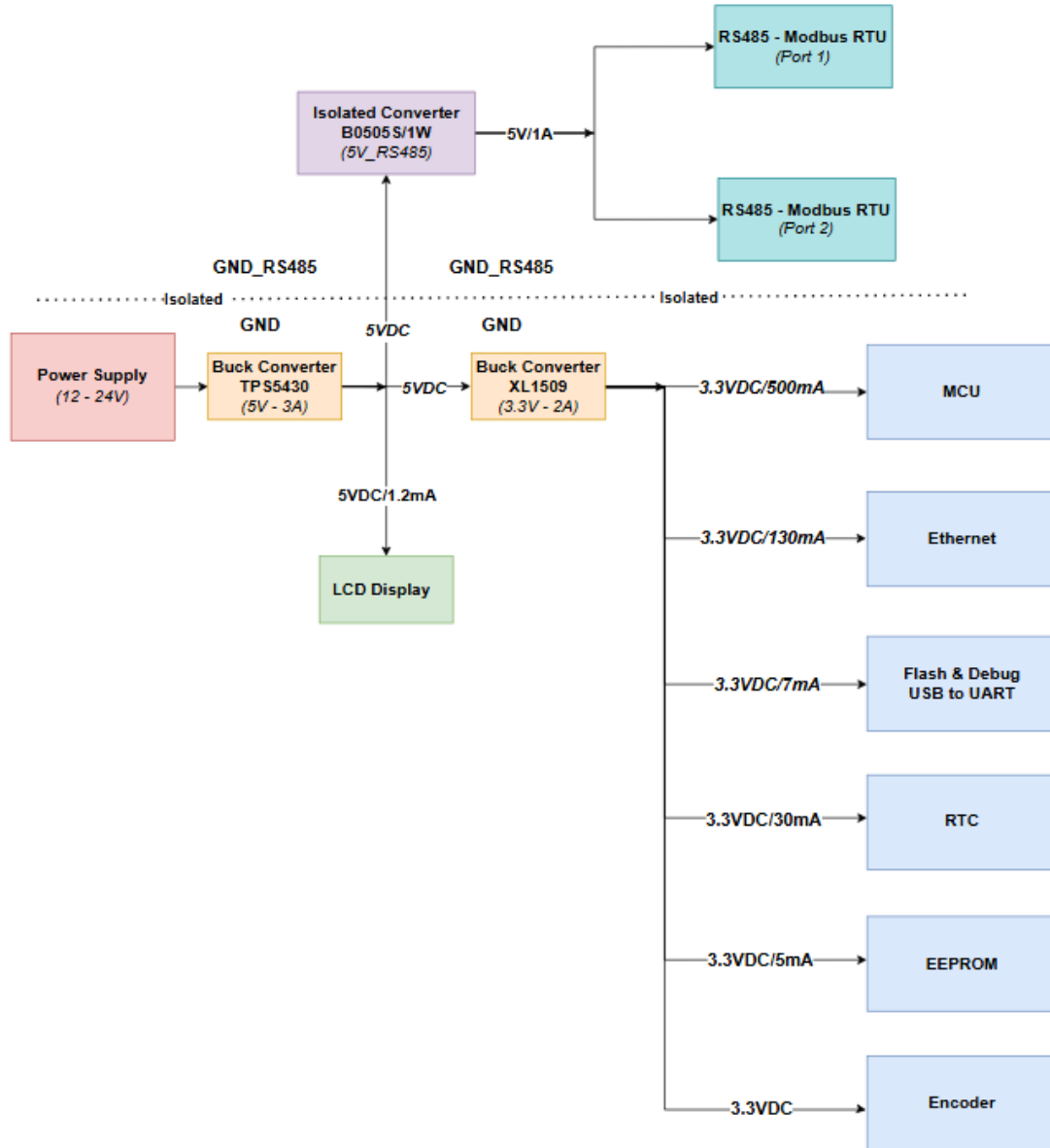
3.1 Thiết kế phần cứng

3.1.1 Tổng quan kiến trúc hệ thống



Hình 3.1: Sơ đồ khối tổng quát thực hiện phần cứng

Khối nguồn (Power Supply): có vai trò hạ áp từ nguồn một chiều DC 12-24V xuống mức 5V và 3.3V để cung cấp điện cho các thành phần khác trong hệ thống, đảm bảo hoạt động ổn định và an toàn.



Hình 3.2: Sơ đồ khối phân bố nguồn trong hệ thống

Nguồn DC 12-24V được đưa vào module hạ áp (buck converter) sử dụng IC TPS5430 để tạo ra điện áp 5V. Từ nguồn 5V tiếp tục hạ áp xuống 3.3V thông qua IC XL1509 hoặc LDO để cấp cho các các khối khác và vi điều khiển, nguồn 3.3V được sử dụng chủ yếu trong hệ thống nên ta chọn IC XL1509 cho dòng ngõ ra ở khoảng rộng hơn (1-2A) thay vì sử dụng LDO ngõ ra chỉ khoảng 1A.

- **Nguồn 5V:** cấp cho LCD, module RS485.
- **Nguồn 3.3V:** cấp cho vi điều khiển ESP32, module Ethernet, mạch debug cho ESP32, module RTC và EEPROM.

Khối vi điều khiển xử lý trong tâm (Logic Unit): sử dụng ESP32 do có khả năng tích hợp WiFi và BLE Xử lý các tác vụ chính của hệ thống, đảm nhận các nhiệm vụ sau:

- Giao tiếp với các thiết bị ngoại vi như LCD, module RS485, RTC và EEPROM thông qua các giao thức SPI, I2C và UART.
- Thu thập dữ liệu từ các thiết bị giám sát (Inverter, Meter, Weather Station) qua giao thức Modbus RTU qua RS485.
- Xử lý dữ liệu thu thập được và truyền về Server thông qua giao thức Modbus TCP/IP qua module Ethernet.
- Giao tiếp với web hiển thị dữ liệu thông qua giao thức MQTT.

Khối RTC (Real-Time Clock): sử dụng module DS3231 để cung cấp thời gian chính xác cho hệ thống, hỗ trợ việc đánh dấu thời gian cho dữ liệu thu thập và đồng bộ hóa với Server.

Khối EEPROM: sử dụng module 24C256 để lưu trữ dữ liệu tạm thời khi mất kết nối mạng hoặc khi cần lưu trữ các thông số cấu hình của hệ thống như thông tin kết nối wifi,...

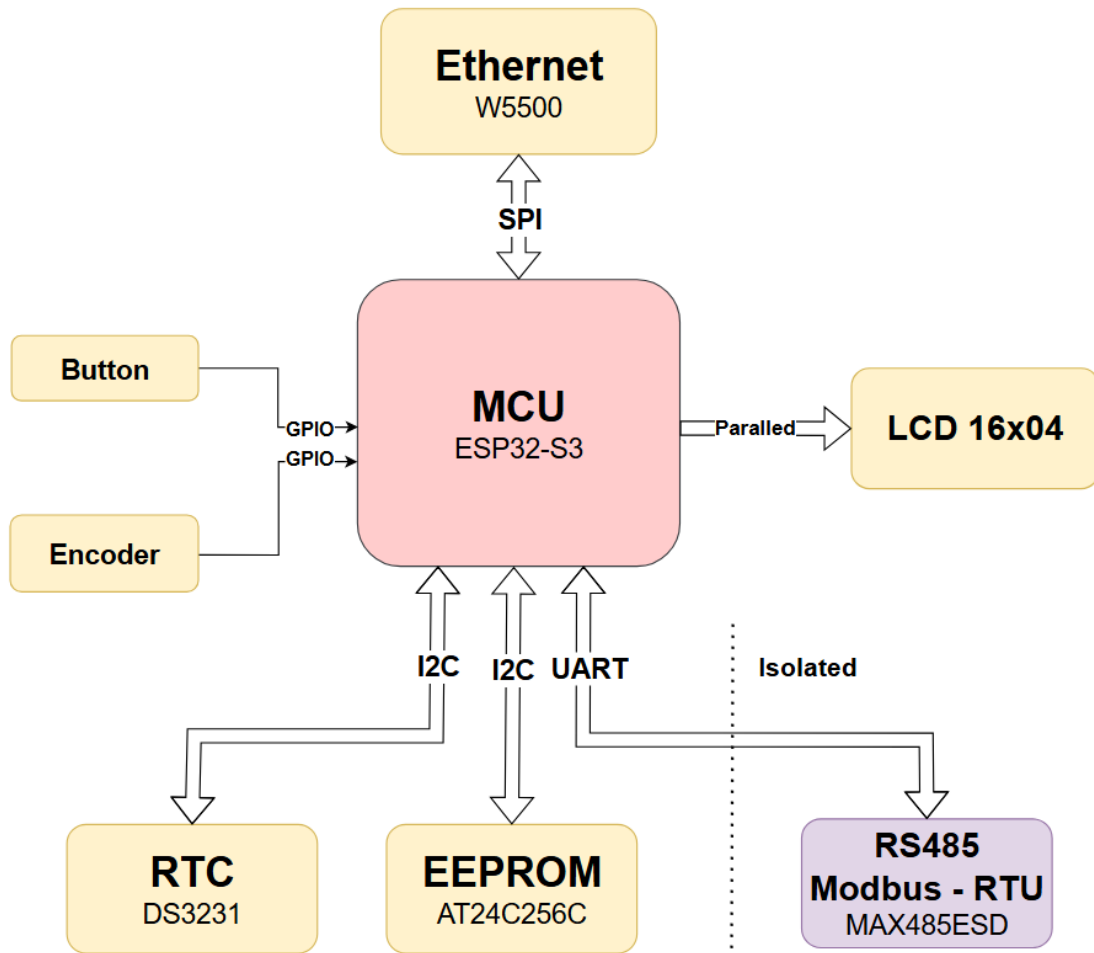
Khối giao tiếp (Communication Unit): bao gồm các giao thức truyền thông:

- Giao tiếp Modbus RTU qua RS485: để thu thập dữ liệu từ các thiết bị giám sát như Inverter, Meter và Weather Station. Giao thức này được lựa chọn do tính ổn định và khả năng truyền dữ liệu ở khoảng cách xa trong môi trường công nghiệp.
- Giao tiếp Modbus TCP/IP qua Ethernet: để truyền dữ liệu đã thu thập về Server. Giao thức này cho phép tích hợp dễ dàng với hệ thống mạng hiện có và hỗ trợ truyền dữ liệu nhanh chóng, đồng thời có thể sử dụng để nhận lệnh lấy dữ liệu từ máy tính gửi tới.
- Giao tiếp MQTT: sử dụng kết nối wifi 2.4GHz để truyền dữ liệu thời gian thực đến web hiển thị, giúp người dùng có thể theo dõi tình trạng hệ thống một cách trực quan và kịp thời.

Khối giao tiếp với người dùng (User Interface Unit) bao gồm các thành phần:

- LCD 16x04: hiển thị các thông số tổng như tổng công suất, tổng điện áp, tổng dòng điện,... giúp người dùng có thể theo dõi nhanh tình trạng hệ thống ngay tại chỗ.
- Nút nhấn và Encoder: để người dùng có thể tương tác với hệ thống

3.1.2 Các luồng tín hiệu có trong hệ thống



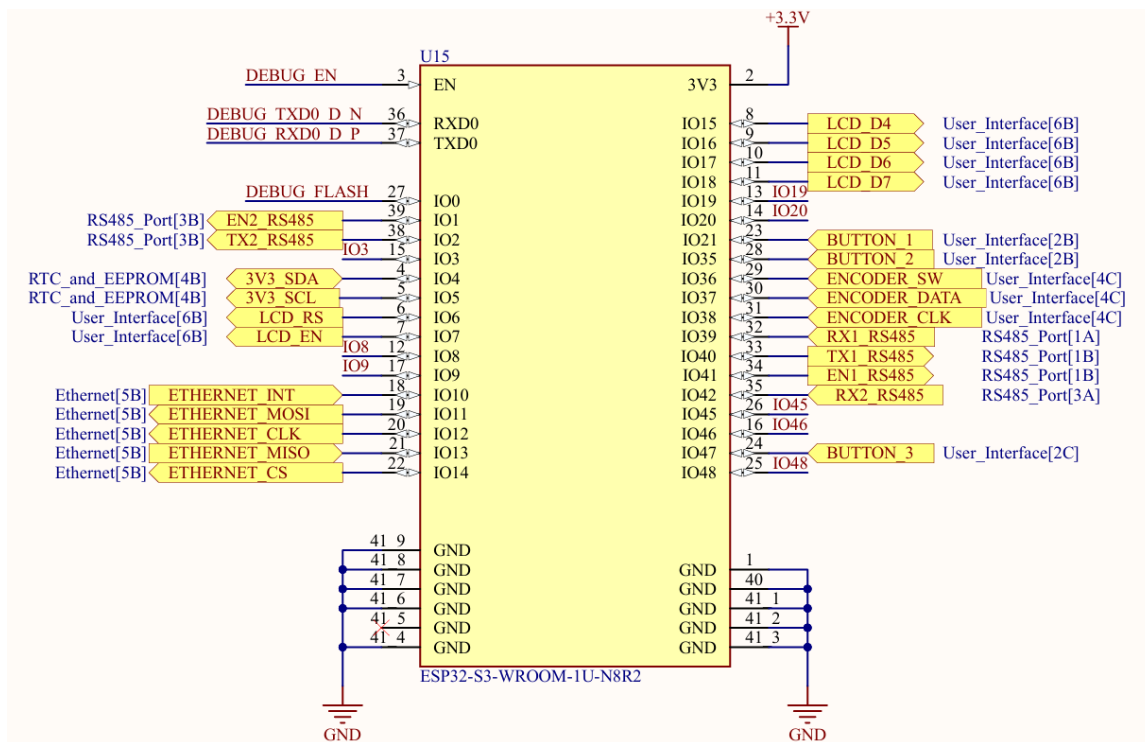
Hình 3.3: Sơ đồ khối luồng dữ liệu trong hệ thống

- Máy tính gửi yêu cầu đọc dữ liệu của thiết bị bất kì bên dưới.
- Dữ liệu từ các thiết bị giám sát (Inverter, Meter, Weather Station) được thu thập định kỳ thông qua giao thức Modbus RTU qua chuẩn vật lý RS485 và được xử lý bởi ESP32.
- Dữ liệu sau khi được xử lý sẽ được truyền về Server thông qua giao thức Modbus TCP/IP qua module Ethernet. Đồng thời, dữ liệu cũng được truyền đến web hiển thị thông qua giao thức MQTT để người dùng có thể theo dõi tình trạng hệ thống một cách trực quan và kịp thời.
- Các thông số tổng như tổng công suất, tổng điện áp, tổng dòng điện,... sẽ được hiển thị trên LCD 16x04 để người dùng có thể theo dõi nhanh tình trạng hệ thống ngay tại chỗ.
- Người dùng có thể tương tác với hệ thống thông qua màn hình LCD hiển thị các thông số và các nút nhấn tùy chỉnh.

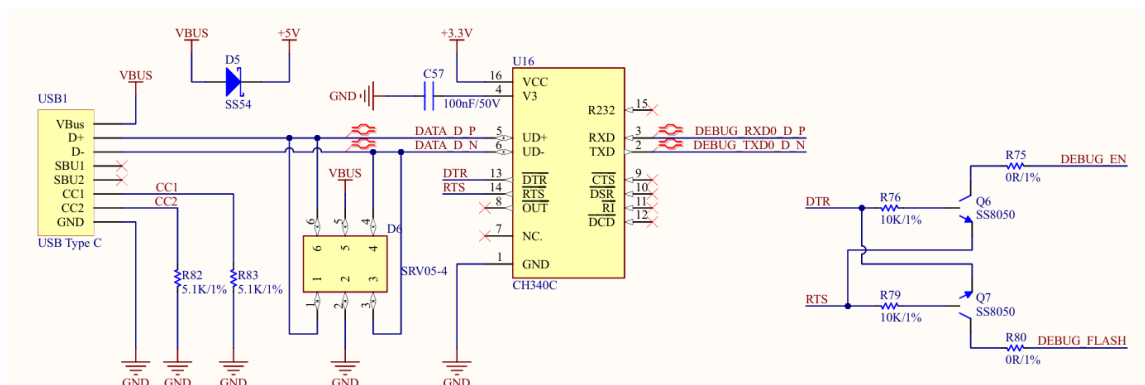
- Các thông số cấu hình như wifi sẽ được lưu trữ ở EEPROM, thiết bị sau khi khởi động lại sẽ đọc các thông số cấu hình từ EEPROM để nạp lại cho hệ thống

3.1.3 Chi tiết thiết kế phần cứng

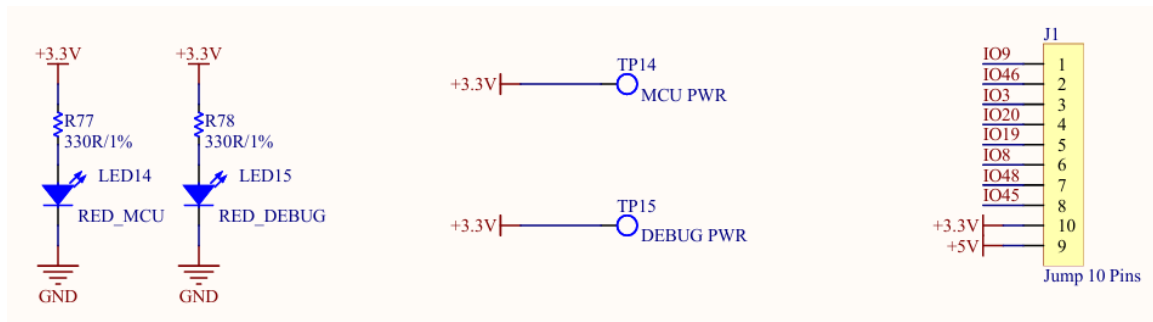
3.1.3.1 Thiết kế khối vi điều khiển



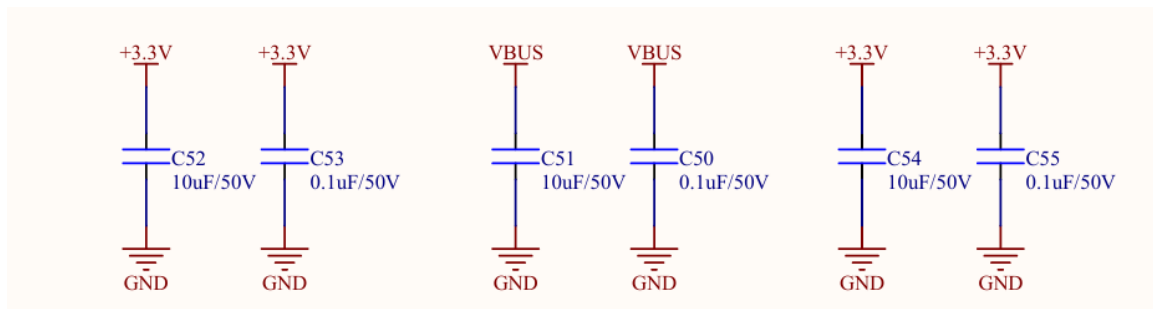
Hình 3.4: khối vi điều khiển



Hình 3.5: khối nạp code và Debug

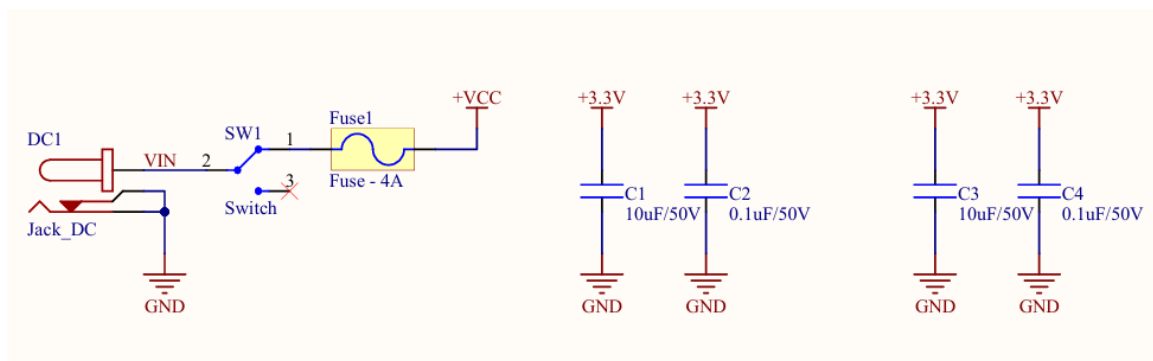


Hình 3.6: khối kiểm tra nguồn cho vi điều khiển

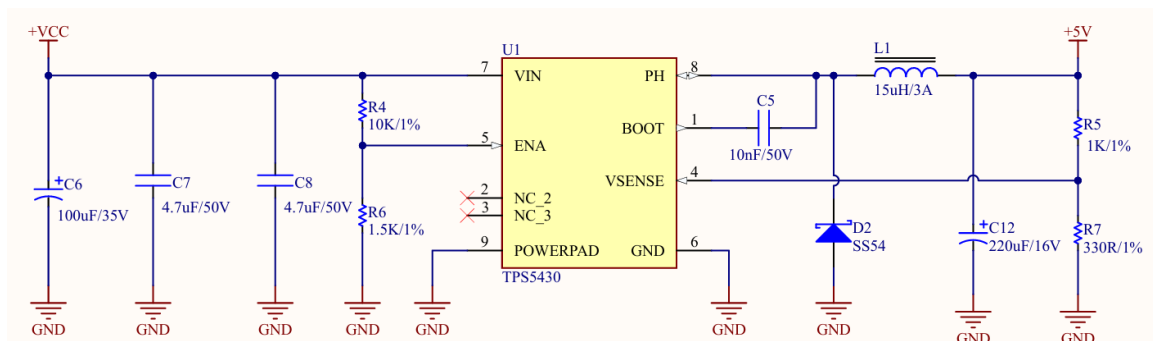


Hình 3.7: khối lọc nguồn đầu vào cho vi điều khiển

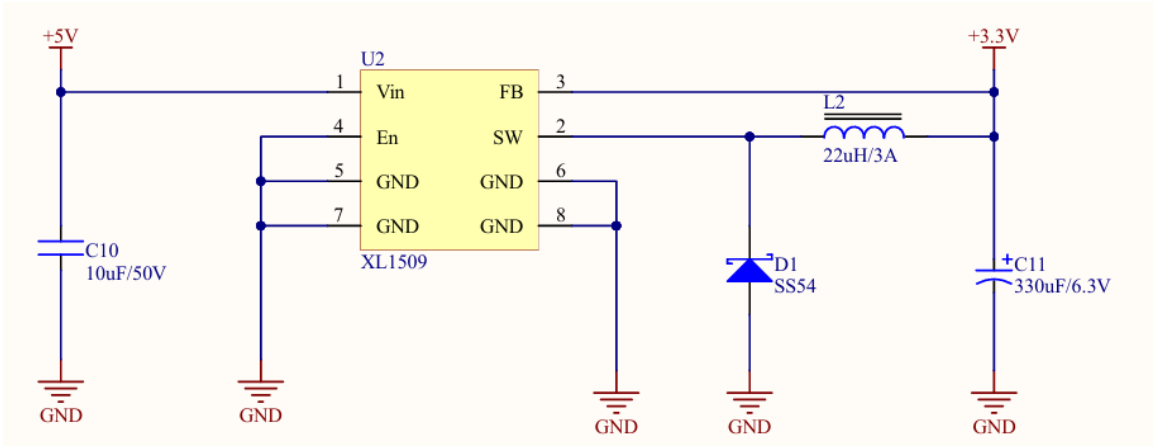
3.1.3.2 Thiết kế khối nguồn



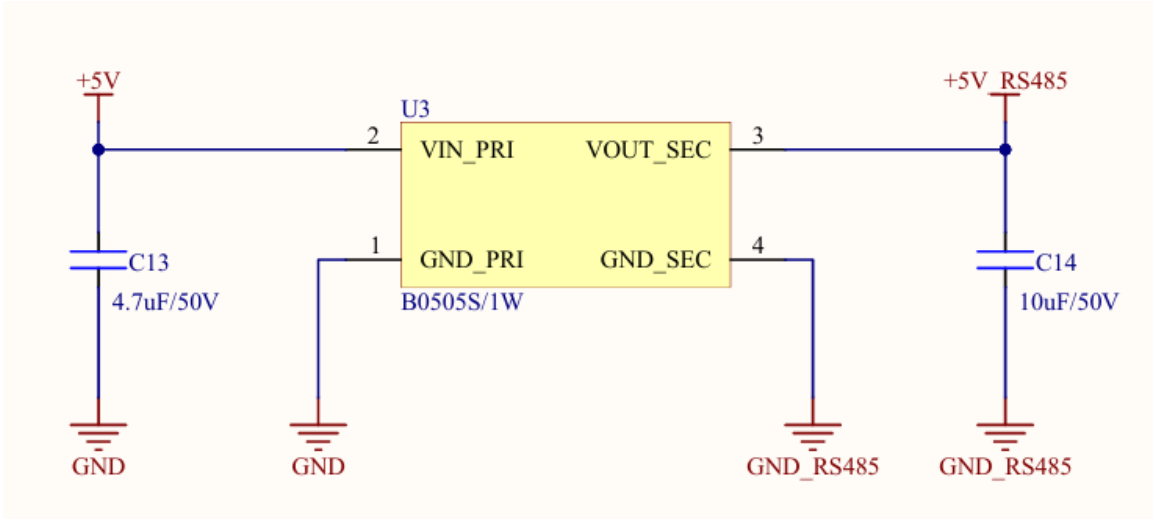
Hình 3.8: khối nguồn đầu vào



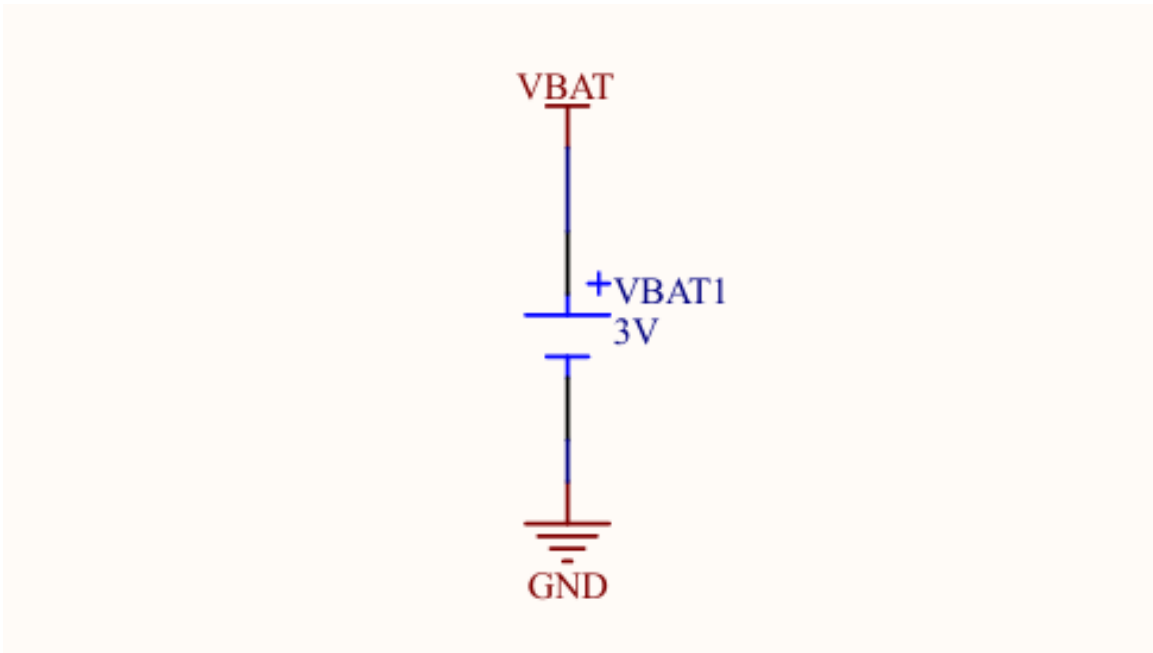
Hình 3.9: Khối nguồn hạ áp 5V



Hình 3.10: Khối nguồn hạ áp 3.3V

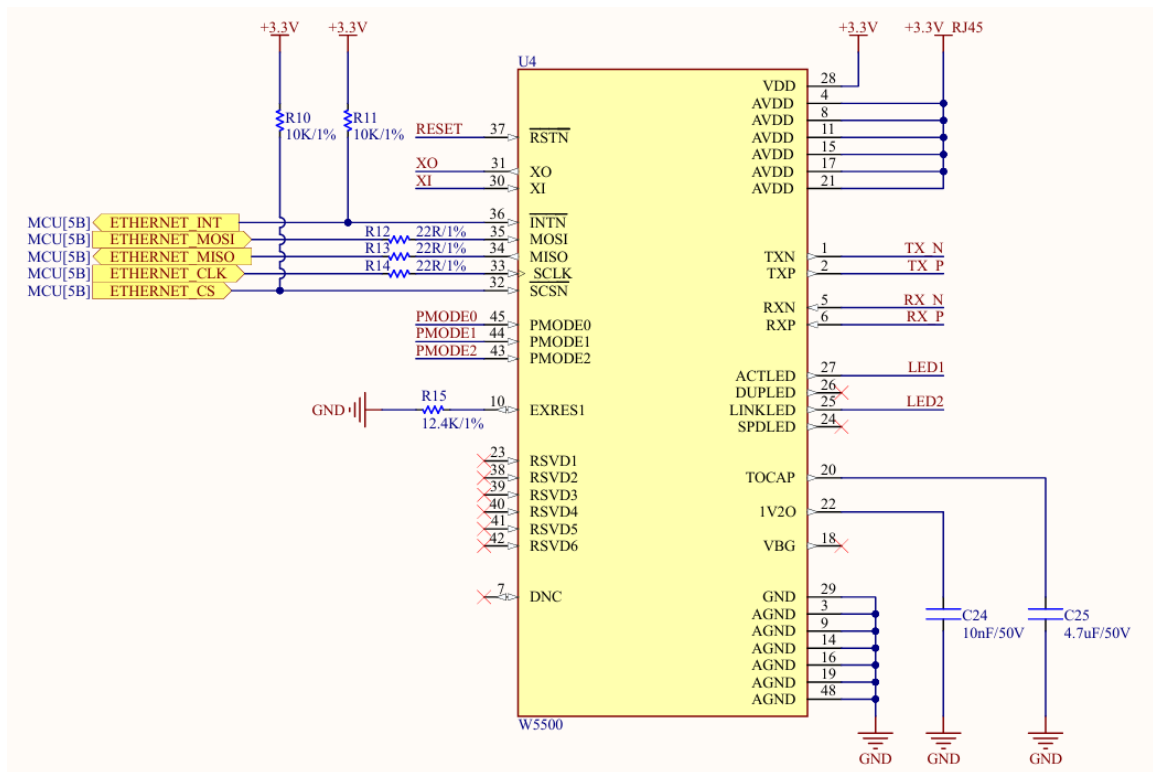


Hình 3.11: khối nguồn cách ly 5V

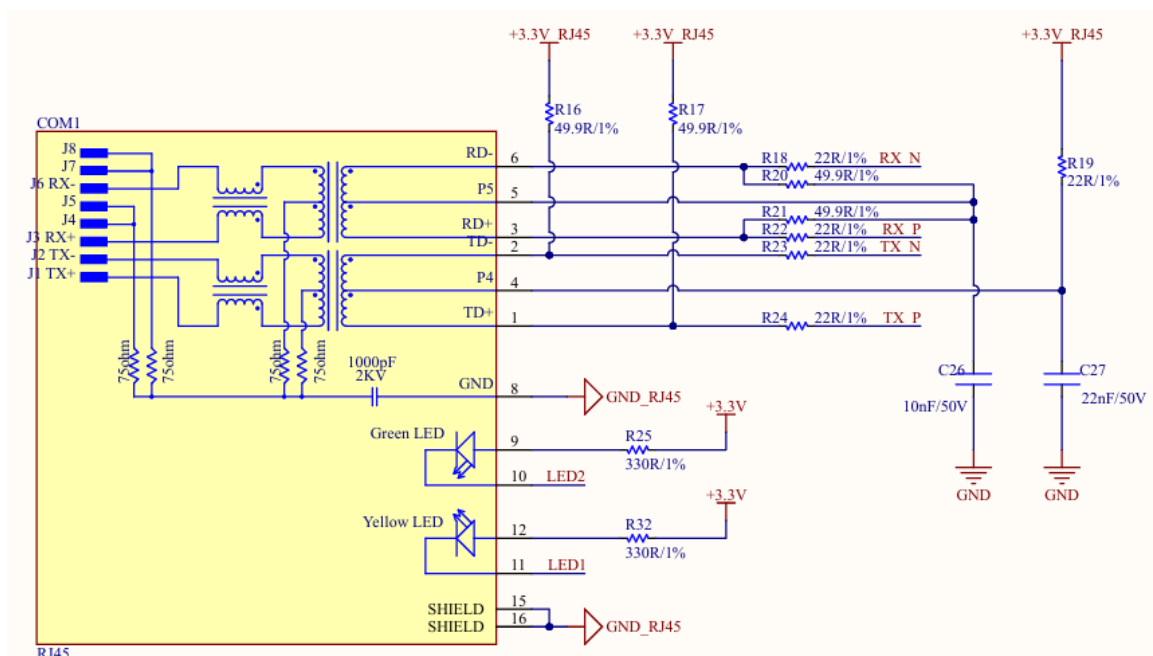


Hình 3.12: khối nguồn Pin rời cho mạch RTC

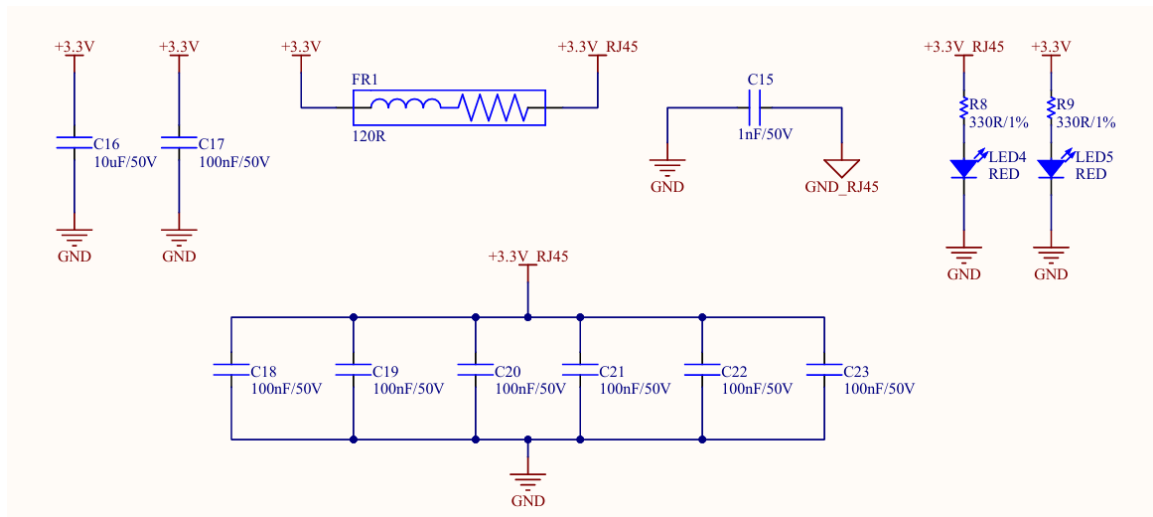
3.1.3.3 Thiết kế khối Ethernet



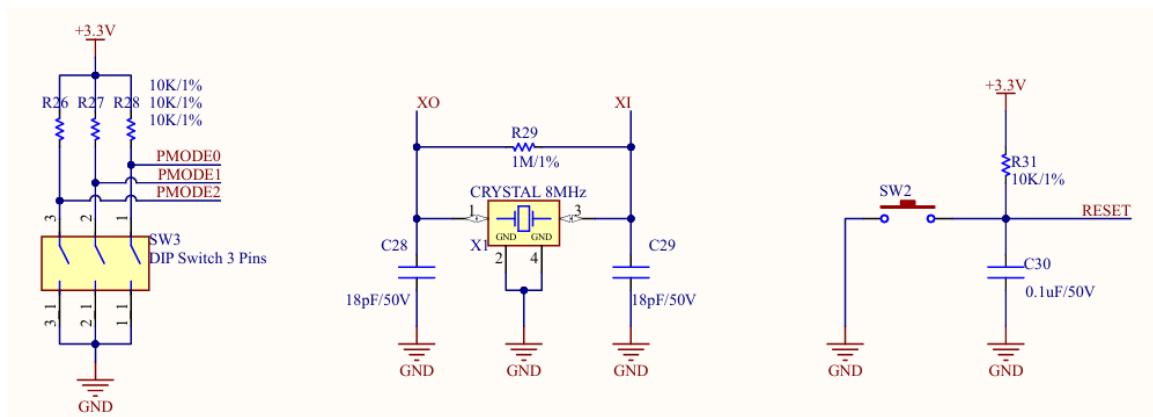
Hình 3.13: khối vi điều khiển cho Ethernet



Hình 3.14: khối cổng kết nối cho Ethernet

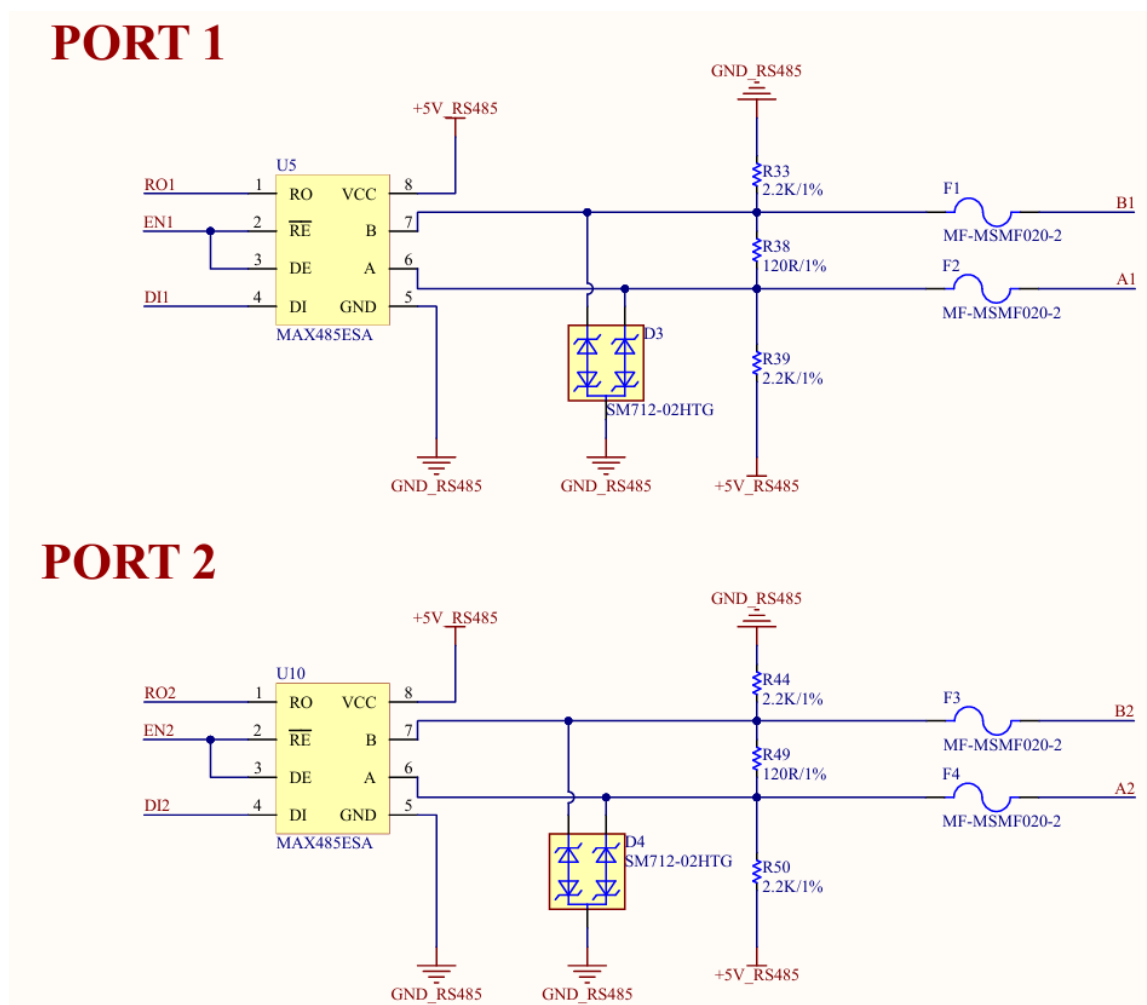


Hình 3.15: khối lọc nguồn đầu vào cho Ethernet

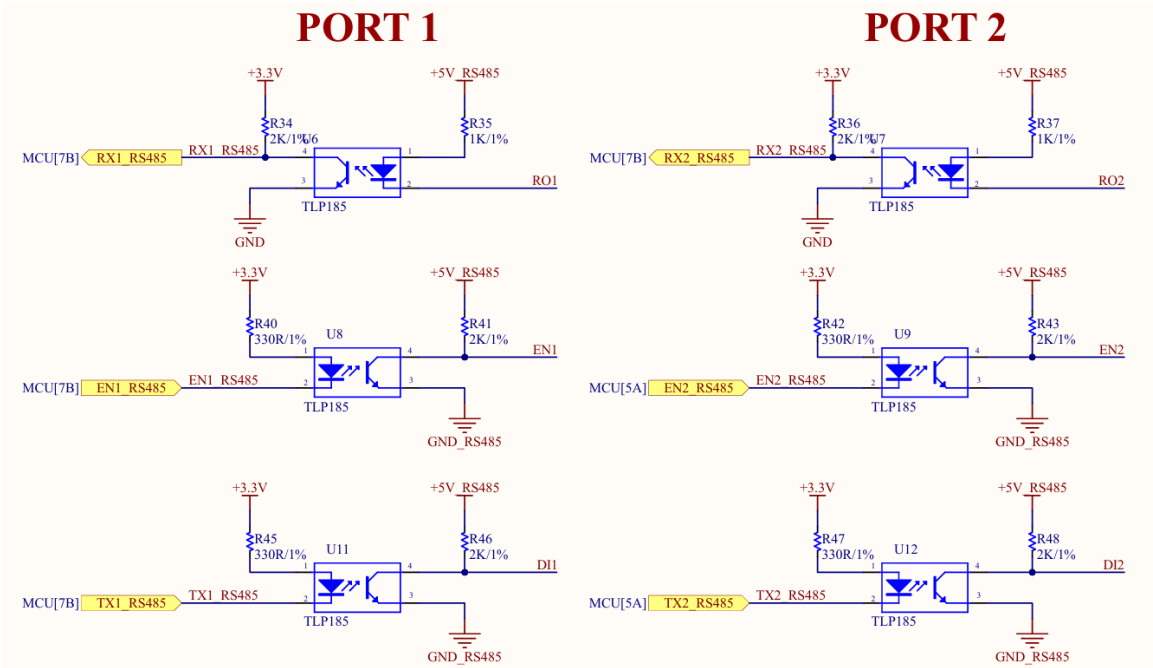


Hình 3.16: khối thạch anh và DIP Switch cho Ethernet

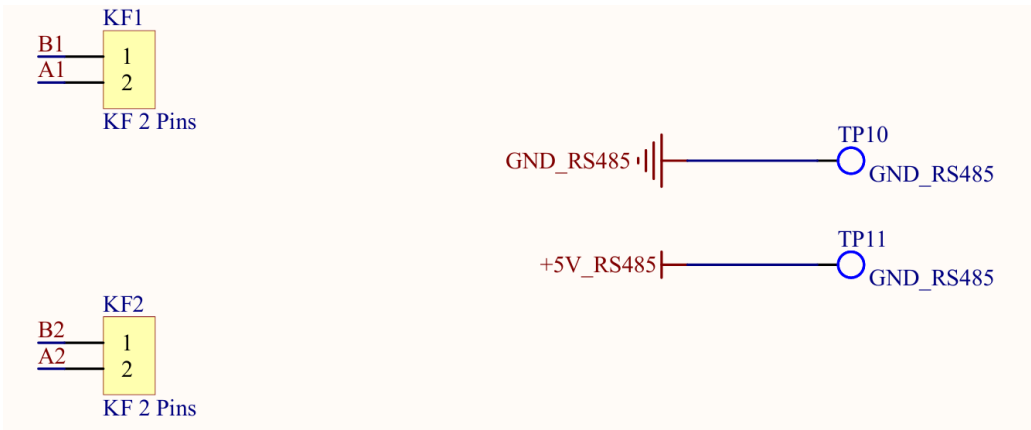
3.1.3.4 Thiết kế khối giao tiếp RS485



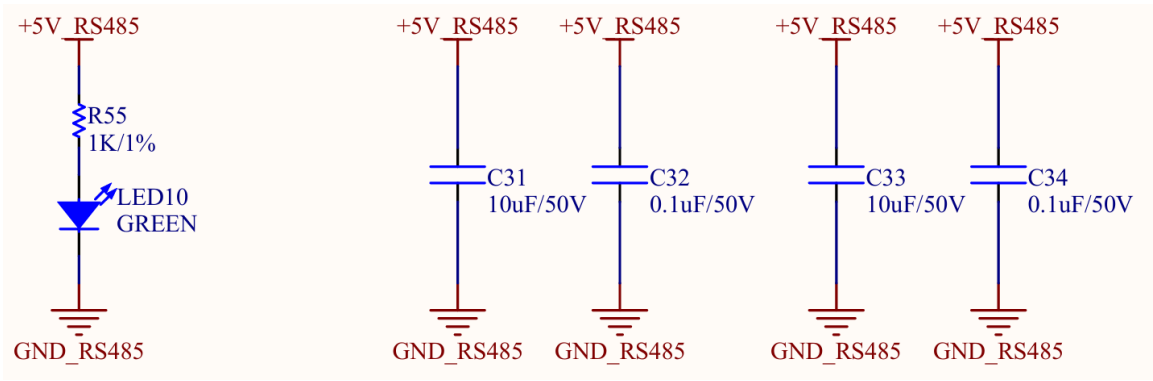
Hình 3.17: khối xử lý tín hiệu RS485



Hình 3.18: khối cách ly RS485

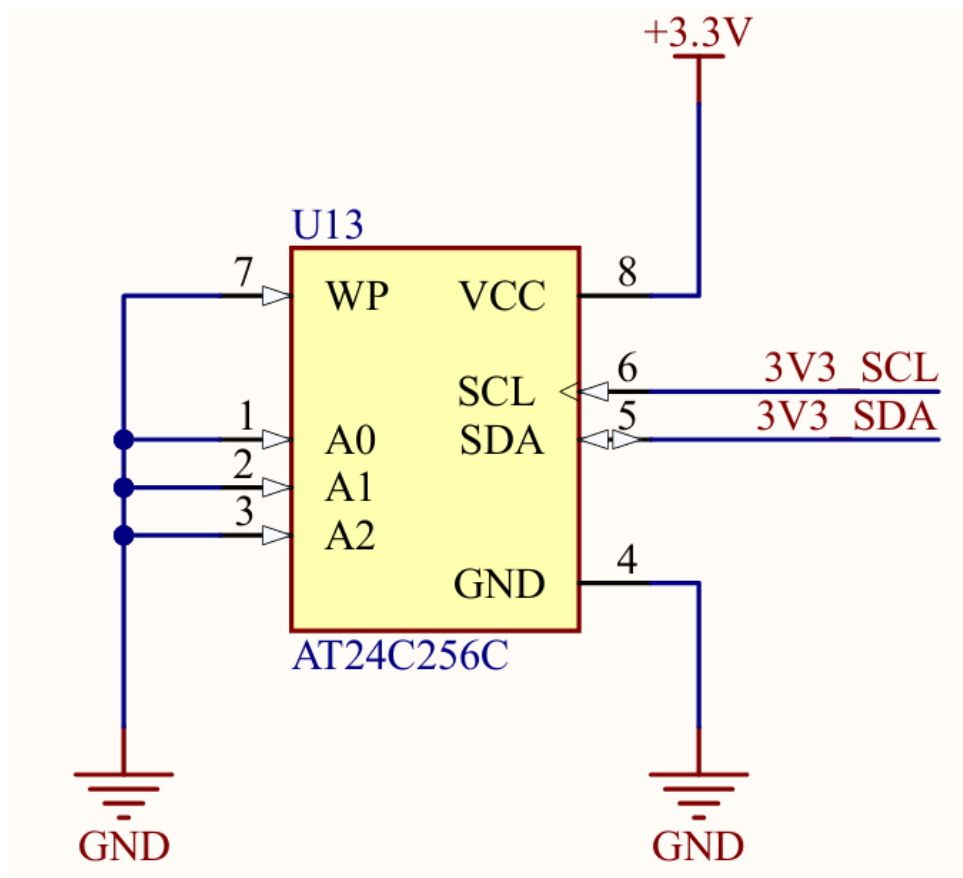


Hình 3.19: khối cổng giao tiếp RS485

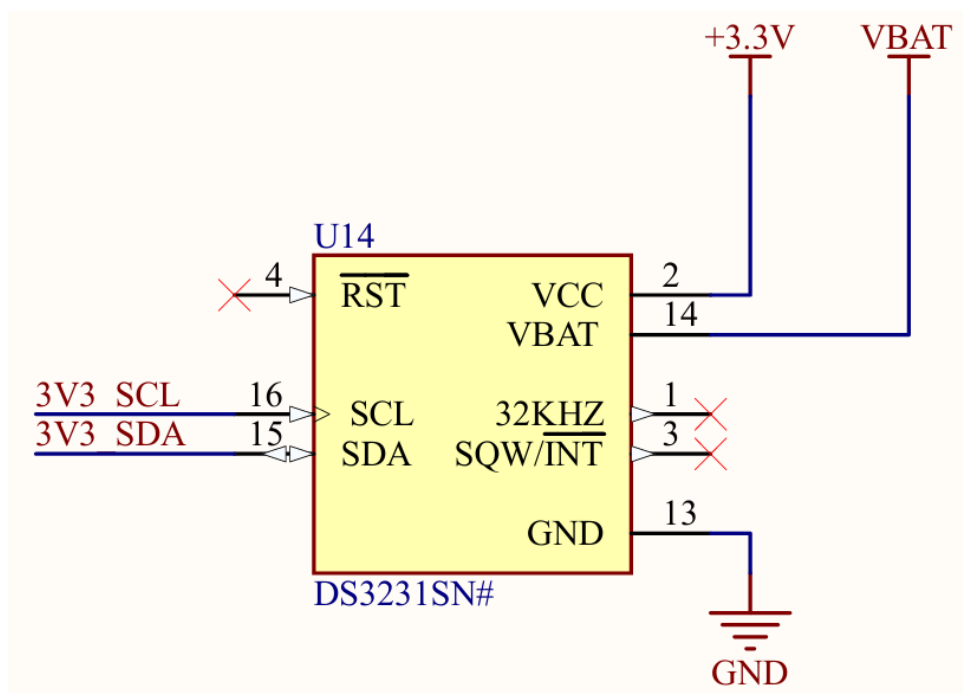


Hình 3.20: khối lọc nguồn đầu vào và LED báo nguồn

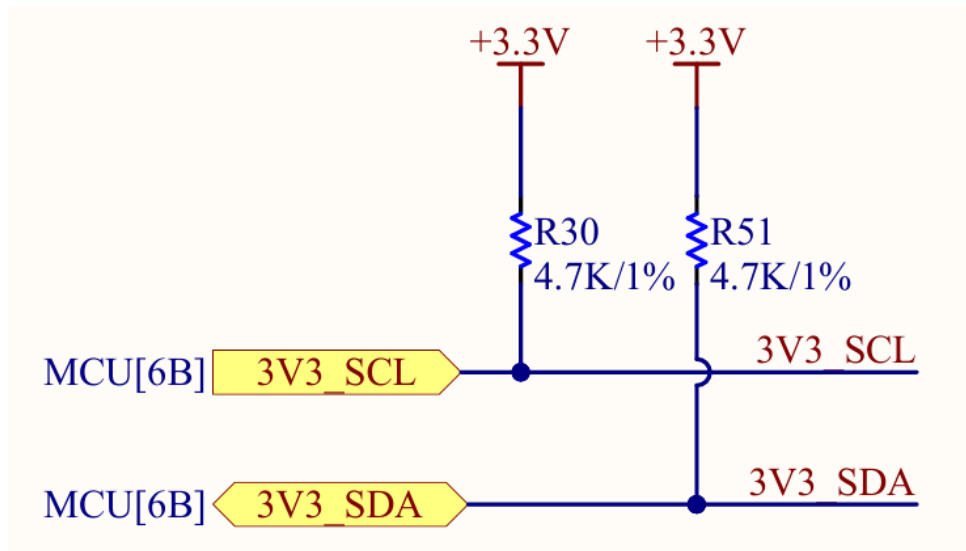
3.1.3.5 Thiết kế khối EEPROM và RTC



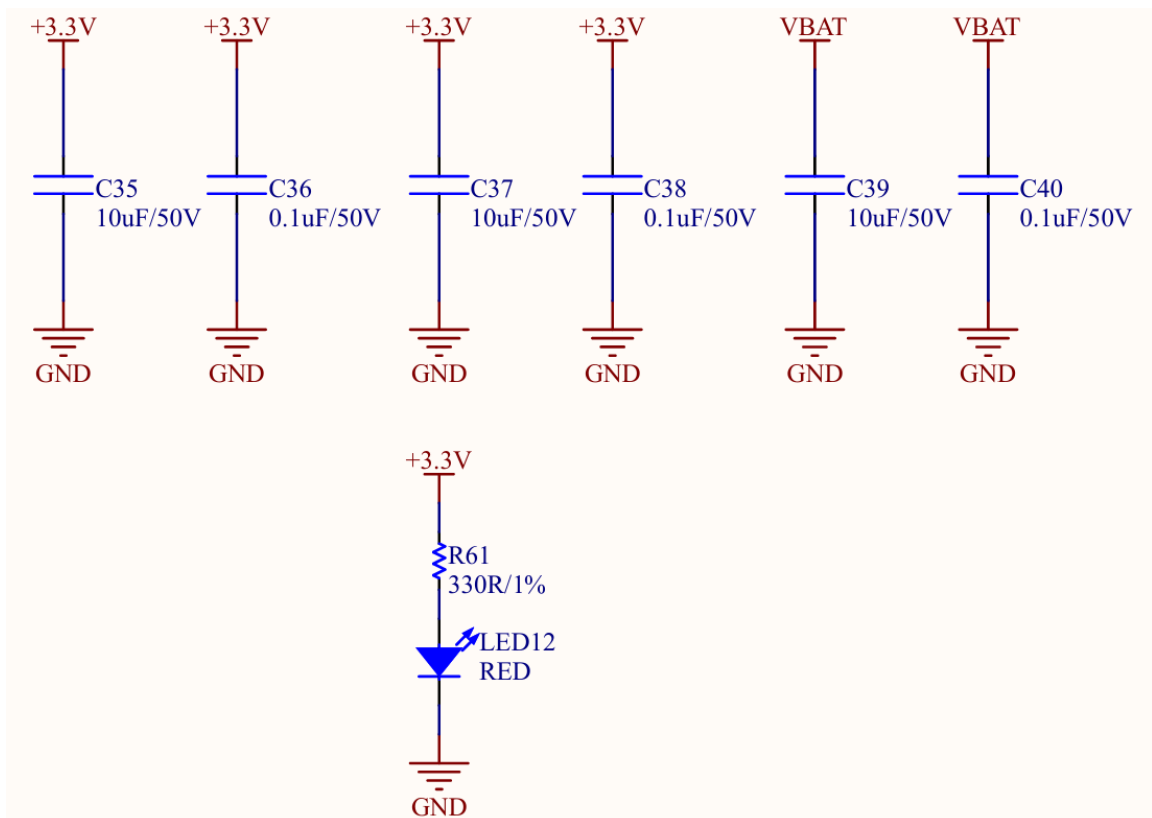
Hình 3.21: khối bộ nhớ EEPROM



Hình 3.22: khối RTC

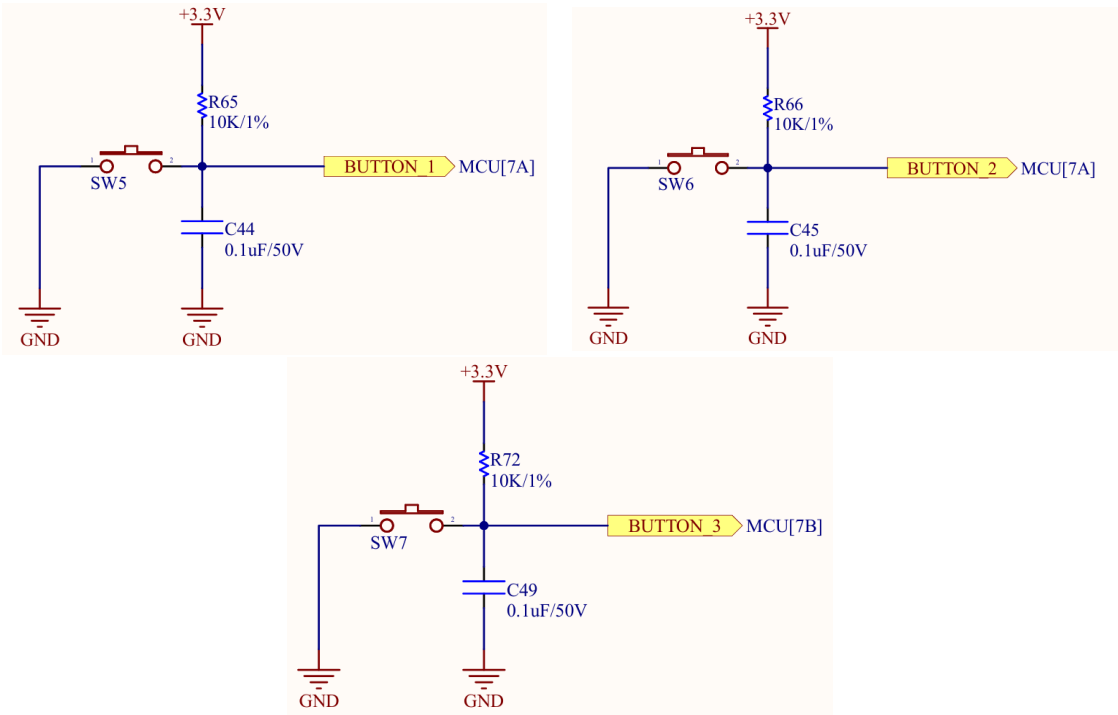


Hình 3.23: khối Pull up cho đường truyền I2C

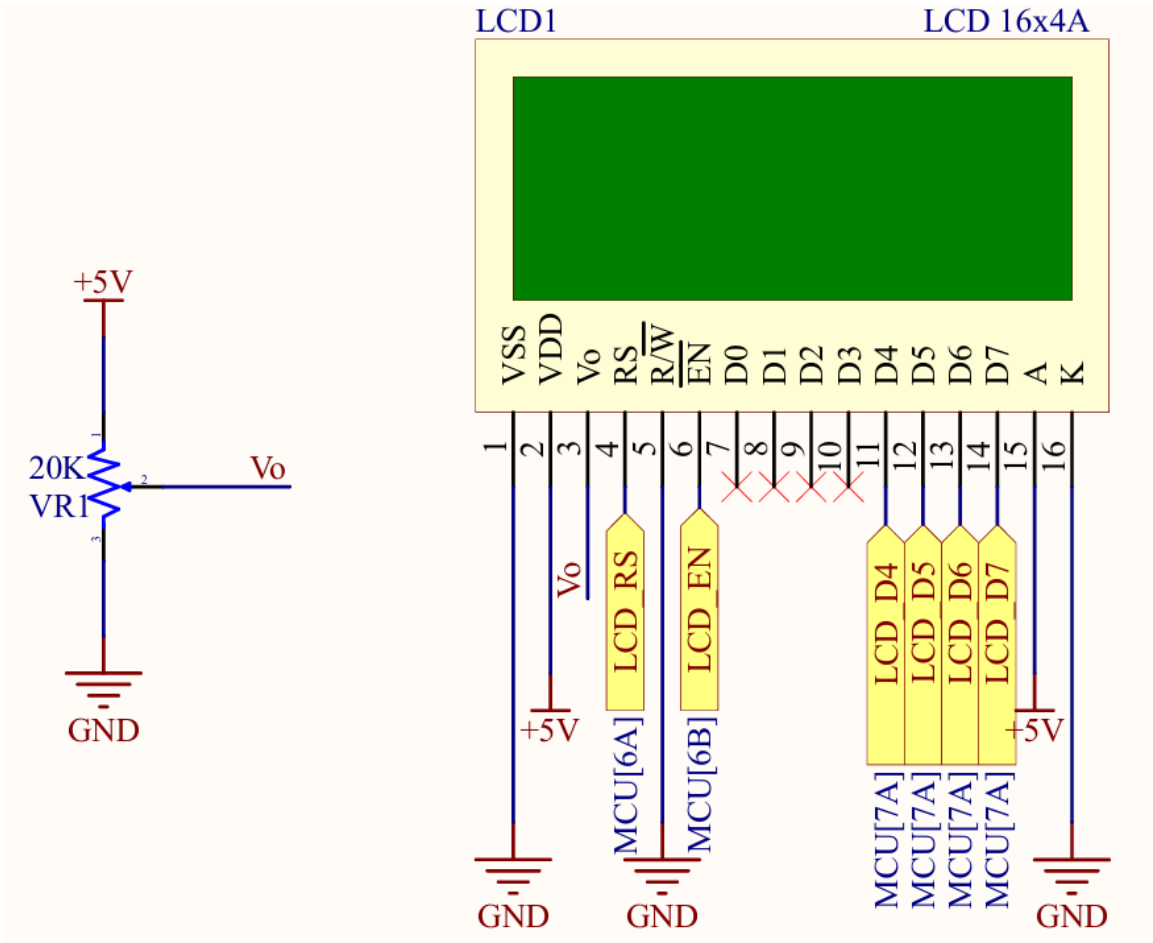


Hình 3.24: khối lọc nguồn đầu vào cho EEPROM và RTC

3.1.3.6 Thiết kế khối giao tiếp với người dùng



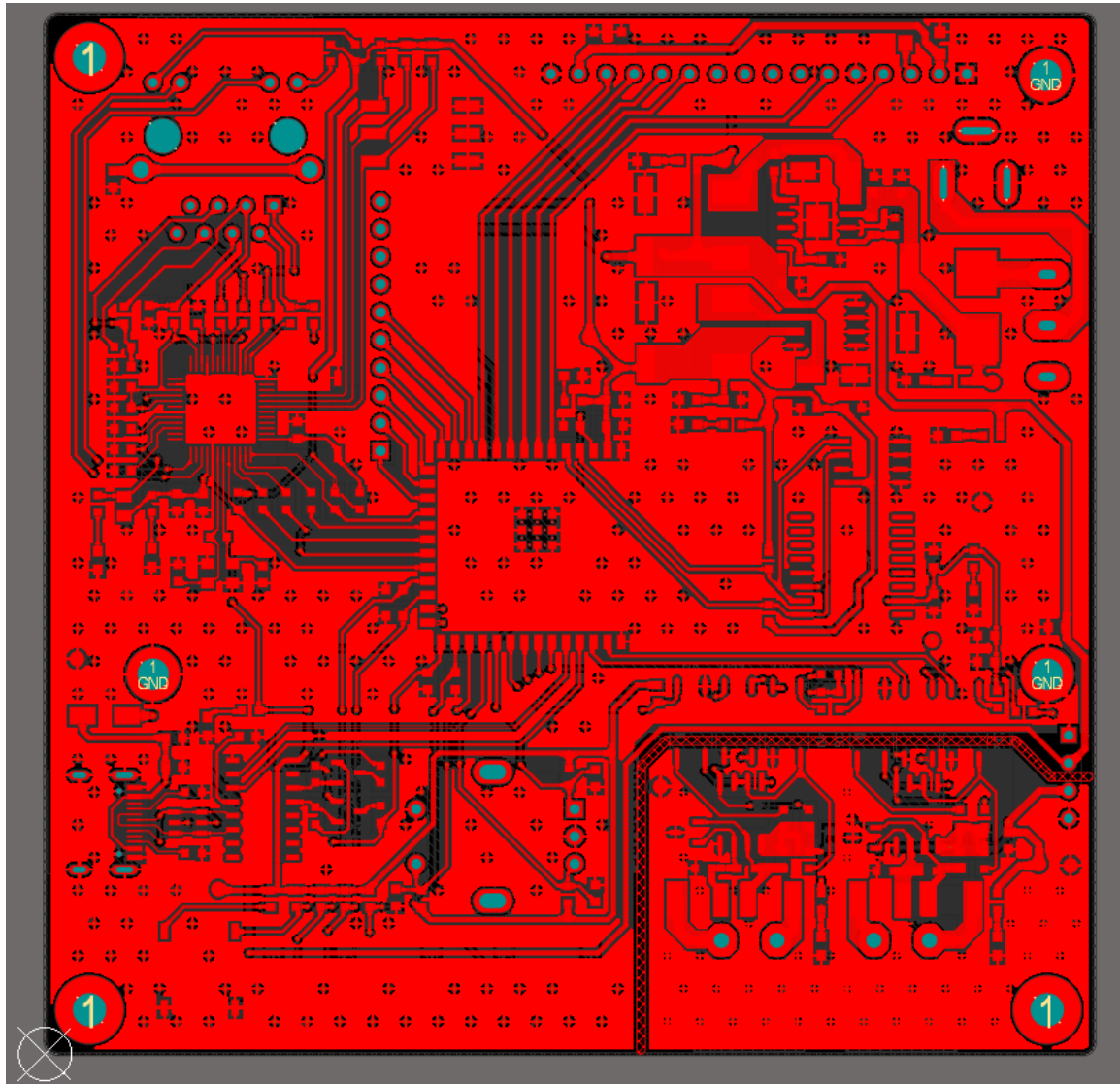
Hình 3.25: khối nút nhấn tương tác với người dùng



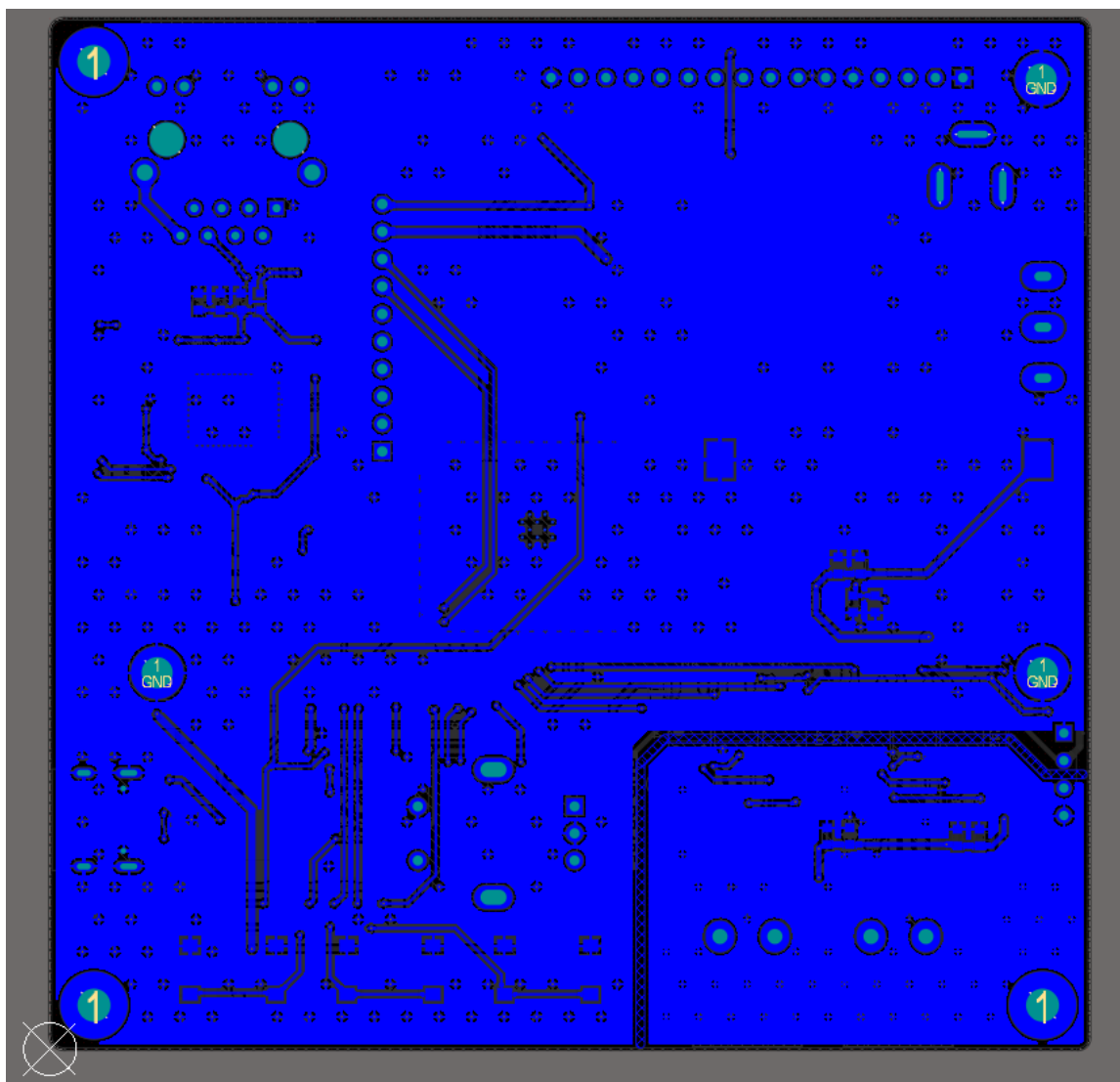
Hình 3.26: màn hình LCD hiển thị

3.1.4 Kết quả thực hiện phần cứng thực tế

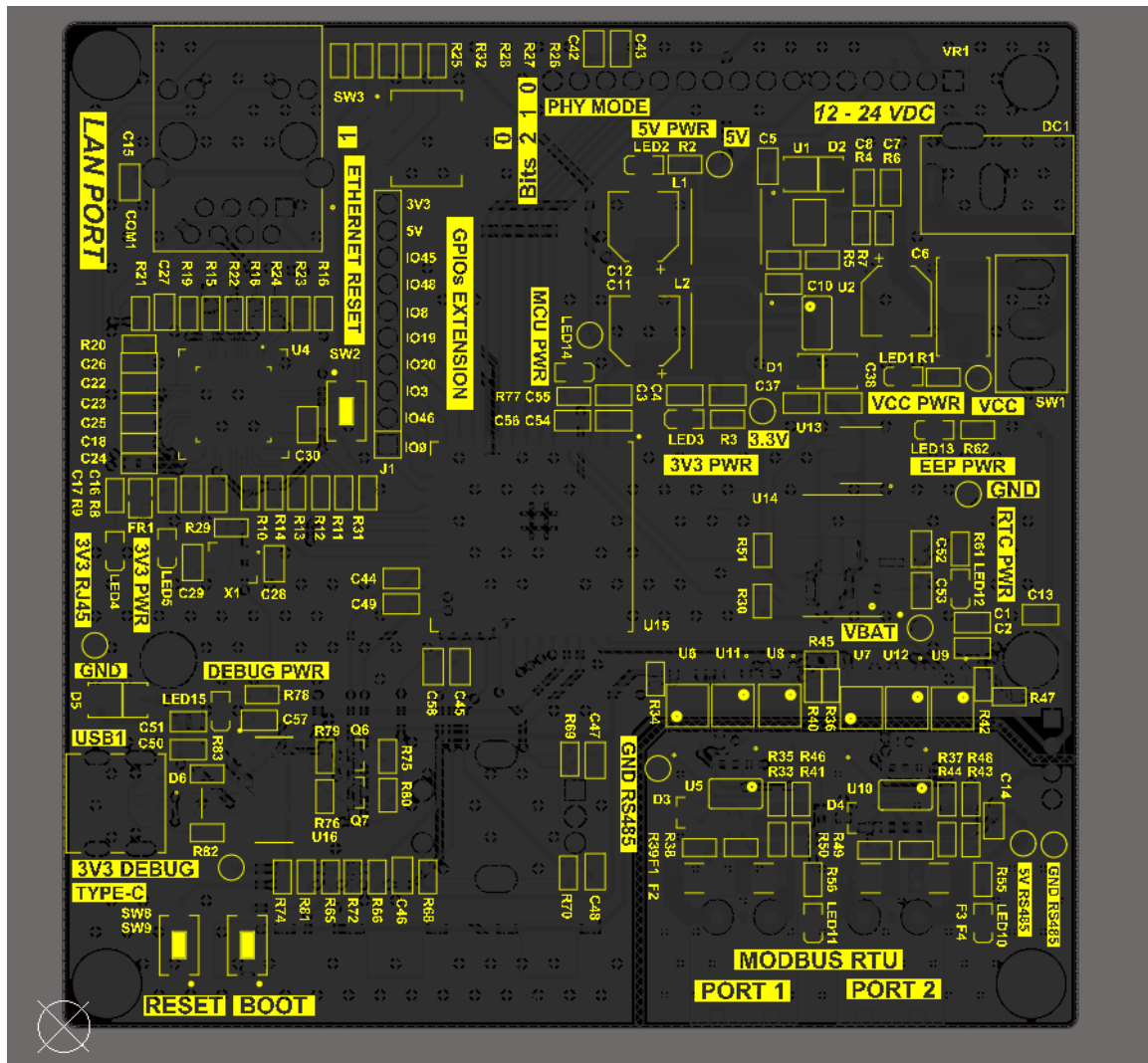
3.1.4.1 Kết quả thiết kế phần cứng trên phần mềm



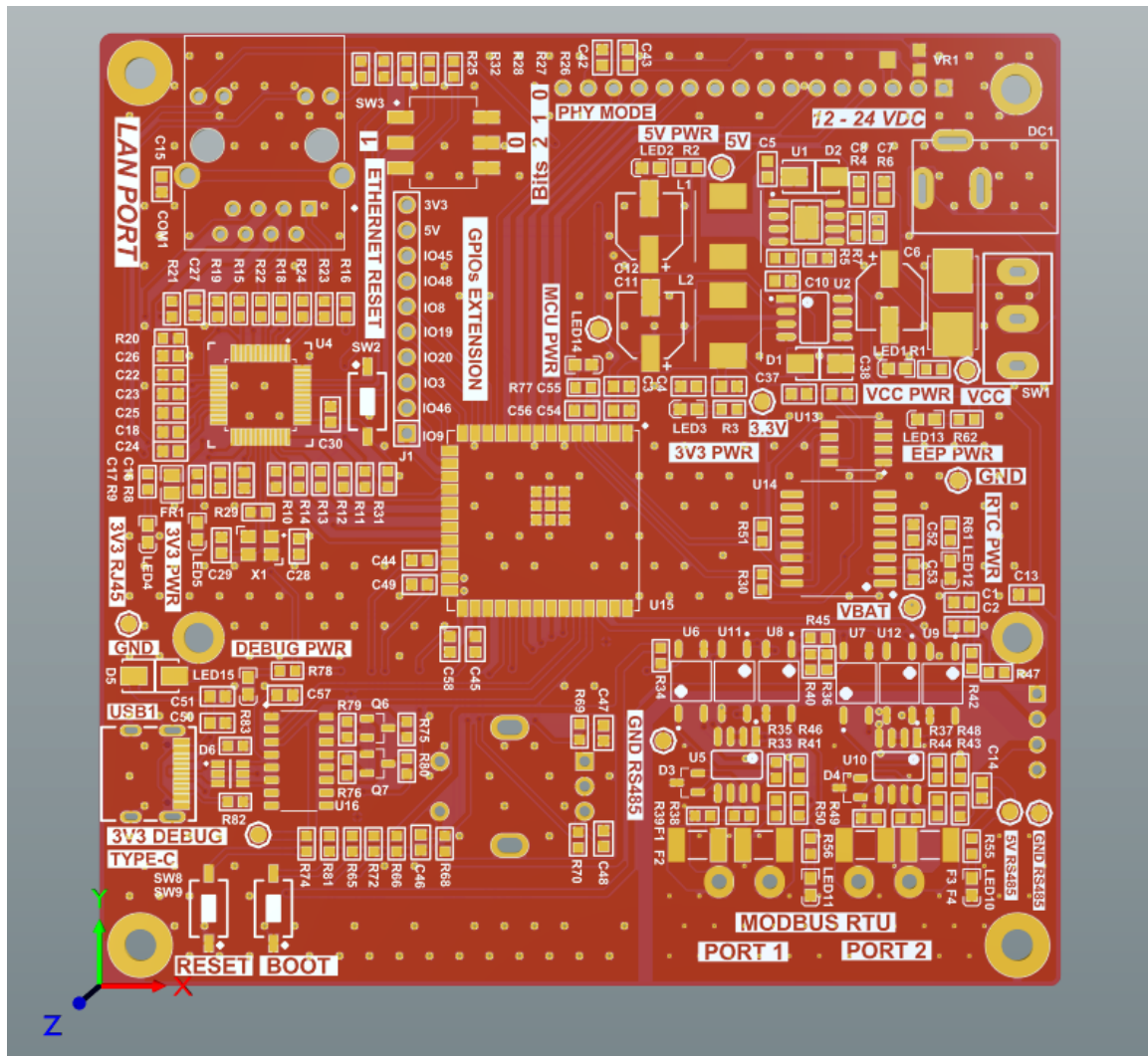
Hình 3.27: phần đi dây ở Top Layer



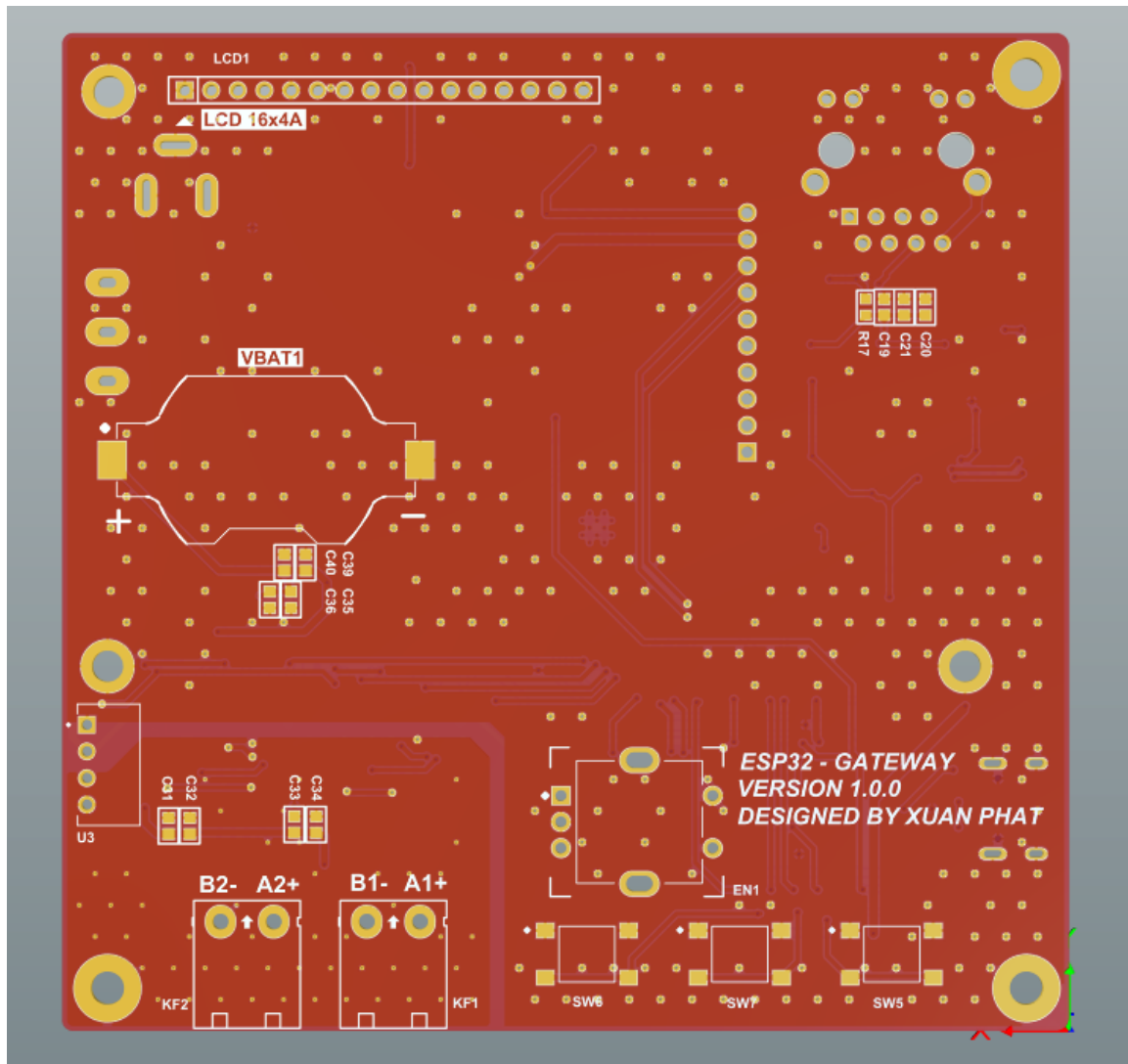
Hình 3.28: phần đi dây ở Bottom Layer



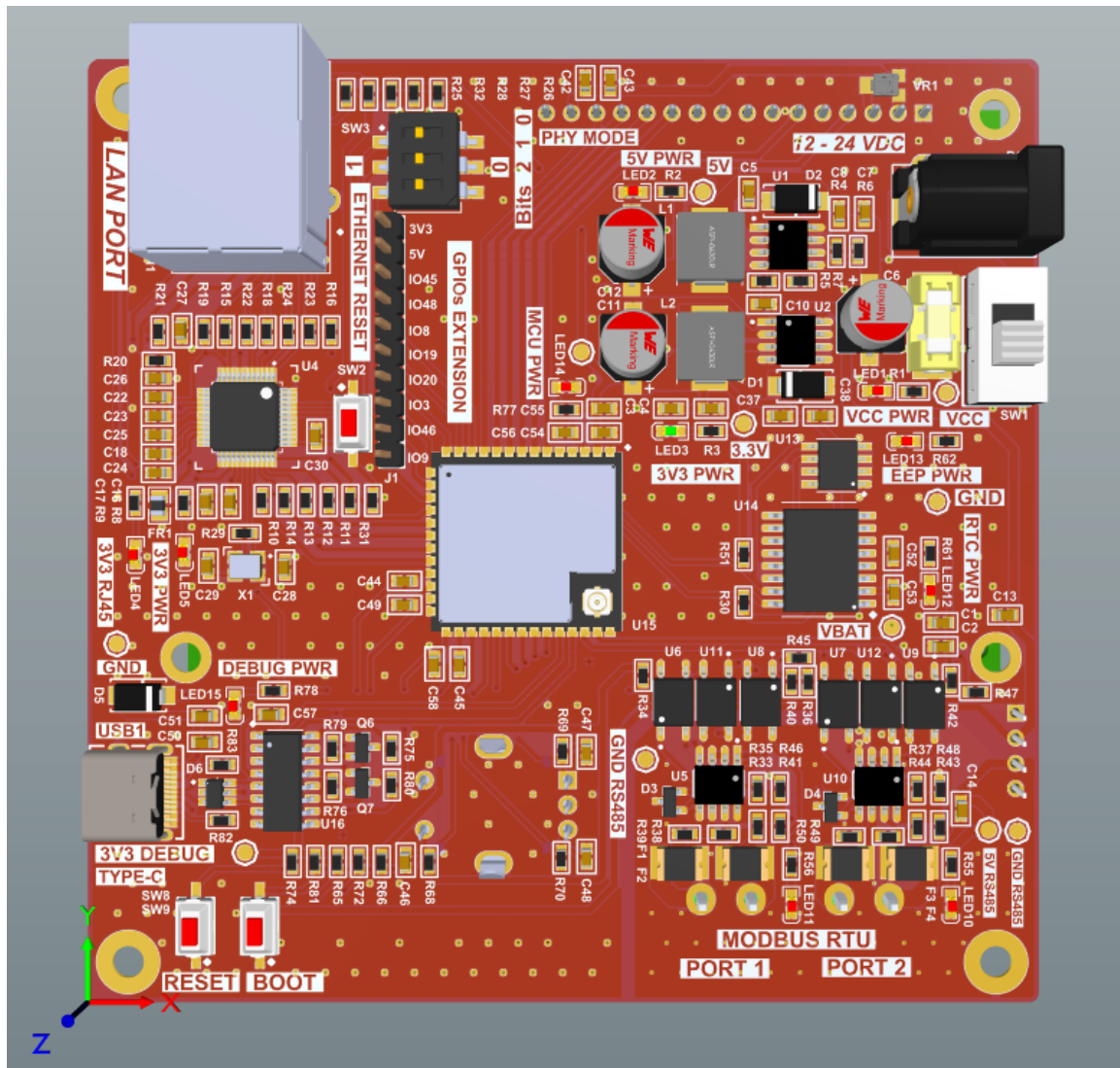
Hình 3.29: phần đặt các symbol ở Top Layer



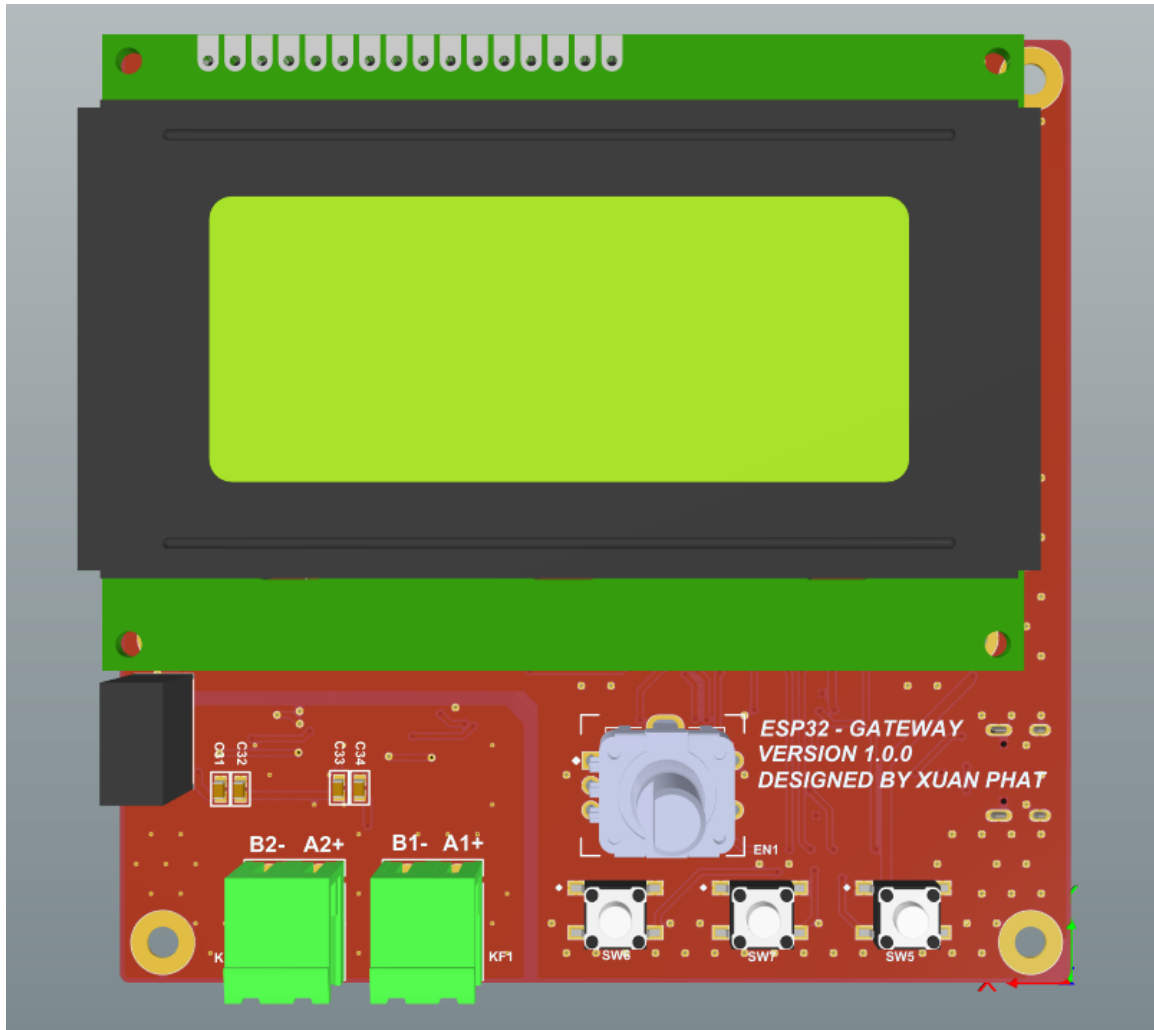
Hình 3.30: mô hình mạch 3D khi chưa đặt linh kiện ở Top Layer



Hình 3.31: mô hình mạch 3D khi chưa đặt linh kiện ở Bottom Layer



Hình 3.32: mô hình mạch 3D khi đặt linh kiện ở Top Layer



Hình 3.33: mô hình mạch 3D khi đặt linh kiện ở Bottom Layer

3.2 Thiết kế phần mềm

3.3 Kết quả thu được

CHƯƠNG 4

Kết luận và hướng phát triển

4.1 Kết luận

4.2 Hướng phát triển